

Contemporary C++: *Learning Modern C++ in a Modern Way*



مدرس: سعيد امراللهی بیوکی

ABOUT ME ...

\$ who am i

- Shahid Beheshti university: BA in Software Engineering
- Azad university: MA in Software Engineering
- 30+ years of experience in C/C++ programming
- I have never had a course on C/C++
- I am the first representative of IRAN at C++ standardization committee at Frankfurt (2009), Rapperswil (2010, 2014), Hawaii (2012) and I paid everything myself.
- In the last 10 years, I have deeply focused on embedded System Programming using C/C++, Programming Instant messengers, and Cloud Gaming.



Agenda 1/30

Getting started: The pre-history of C++, why C++ was created, and a few fundamental concepts

- Course aims and outline
- Is C++ dead?
- Four giant C++ applications: Discovering the Higgs boson, Mars rovers, Human genome sequencing, and the James Webb telescope
- A brief history of C++: Simula, C and System programming
- Software, programming, and programming languages
- C++98: the old but conventional definition
- Q & A

120 min (incl. Q & A)





- It is hard to convince people who do not want to be convinced.
 - Bjarne Stroustrup
- Anyone who stops learning is old, whether at twenty or eighty. Anyone who keeps learning stays young. The greatest thing in life is to keep your mind young.
 - Henry Ford

Is C++ dead?

|s C++ dead?

Is C++ dead?

- 1979-1991

Is C++ dead?

- 1979-1991
- Main reference: Bjarne Stroustrup. Design and Evolution of C++, 1994.

Is C++ dead?

- 1979-1991
- Main reference: Bjarne Stroustrup. Design and Evolution of C++, 1994.

1979

1980

1982

1983

1984

1985

1986

1987

1988

1989

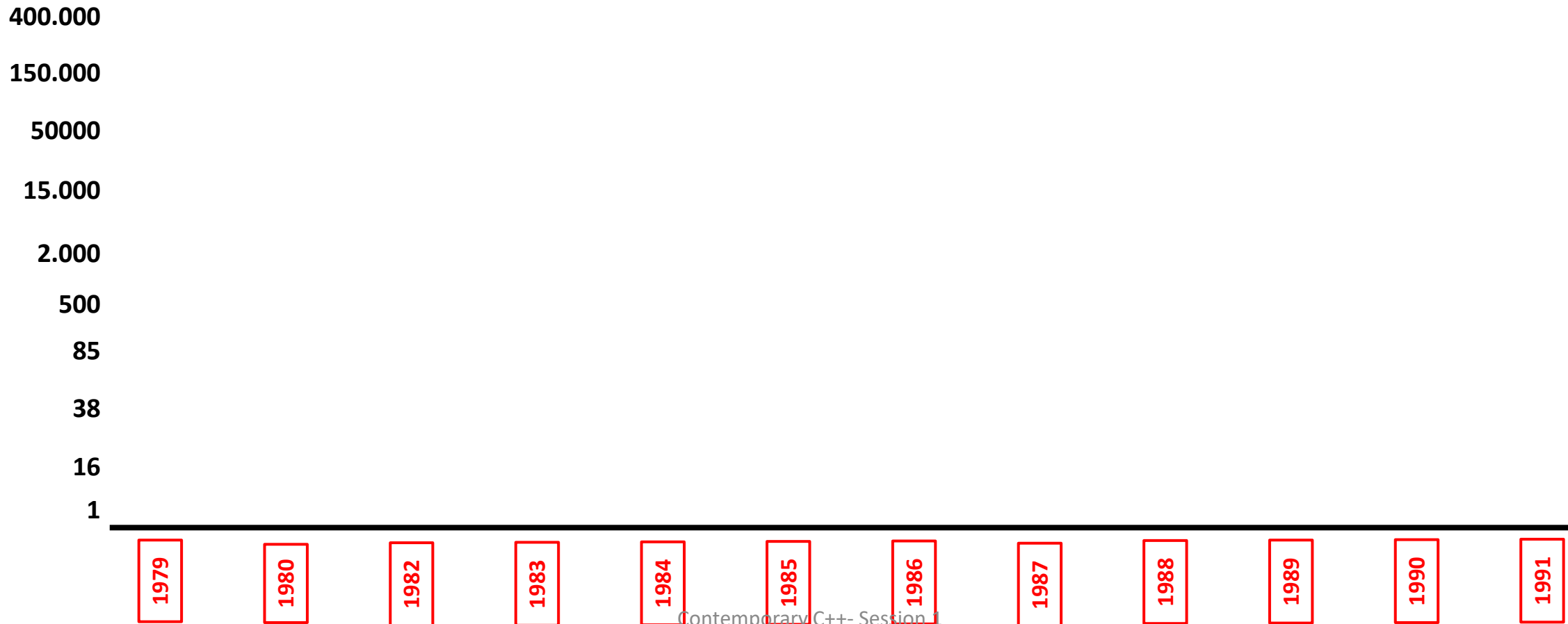
1990

1991

Is C++ dead?

- 1979-1991
- Main reference: Bjarne Stroustrup. Design and Evolution of C++, 1994.

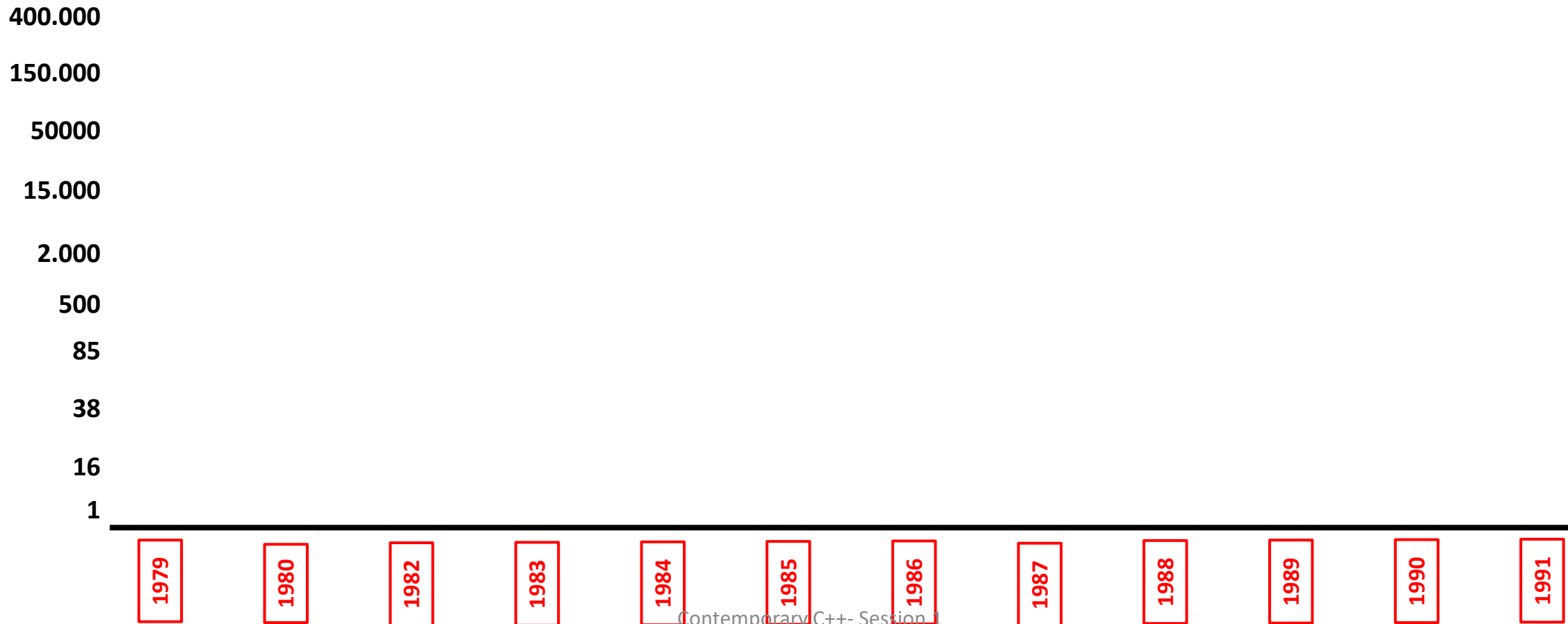
C++ programmers



Is C++ dead?

- 1979-1991
- Main reference: Bjarne Stroustrup. Design and Evolution of C++, 1994.
- During 1980-1991, the number of users doubled every seven and a half months.

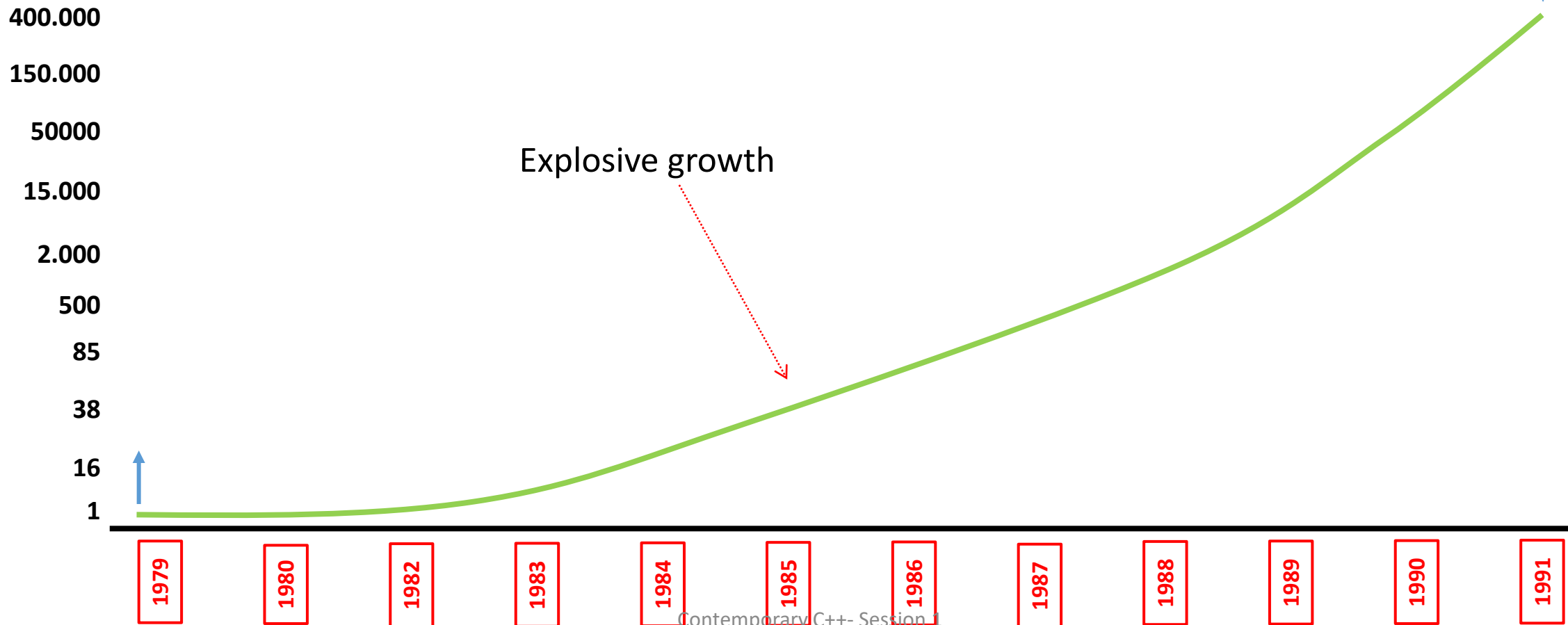
C++ programmers



Is C++ dead?

- 1979-1991
- Main reference: Bjarne Stroustrup. Design and Evolution of C++, 1994.
- During 1980-1991, the number of users doubled every seven and a half months.

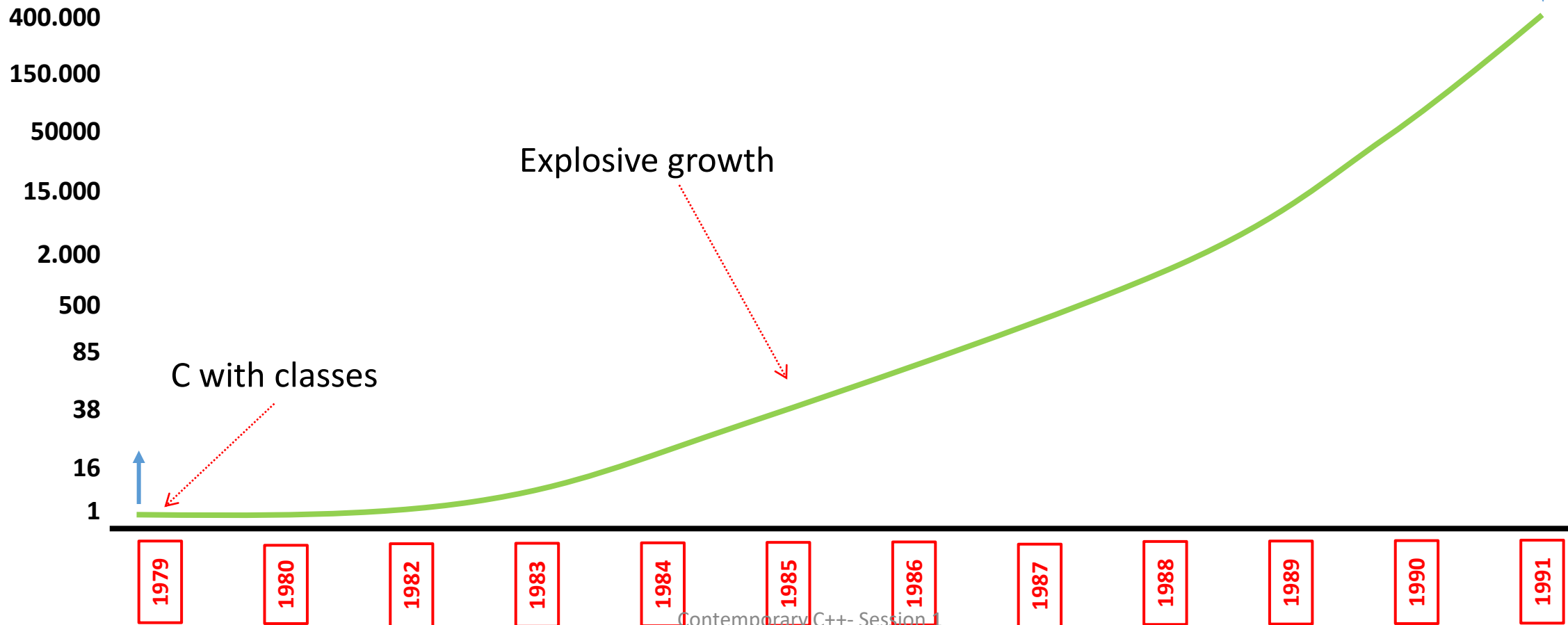
C++ programmers



Is C++ dead?

- 1979-1991
- Main reference: Bjarne Stroustrup. Design and Evolution of C++, 1994.
- During 1980-1991, the number of users doubled every seven and a half months.

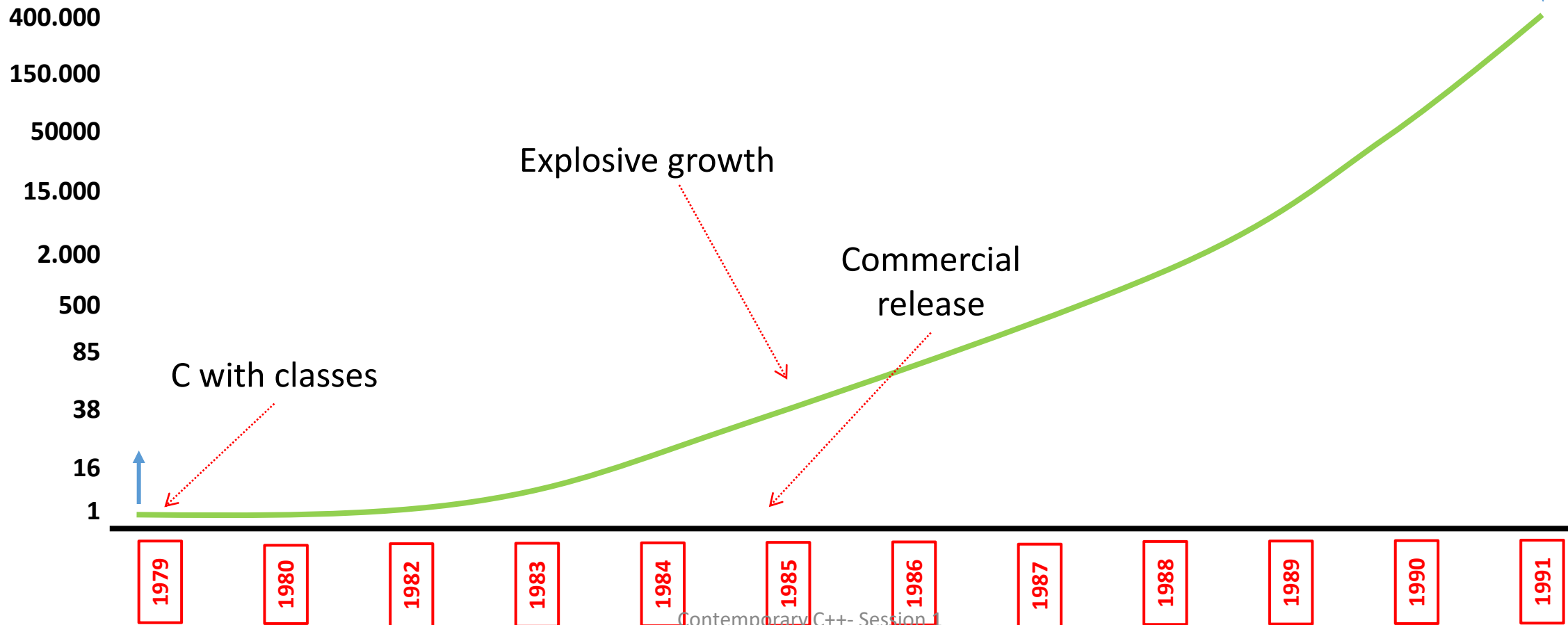
C++ programmers



Is C++ dead?

- 1979-1991
- Main reference: Bjarne Stroustrup. Design and Evolution of C++, 1994.
- During 1980-1991, the number of users doubled every seven and a half months.

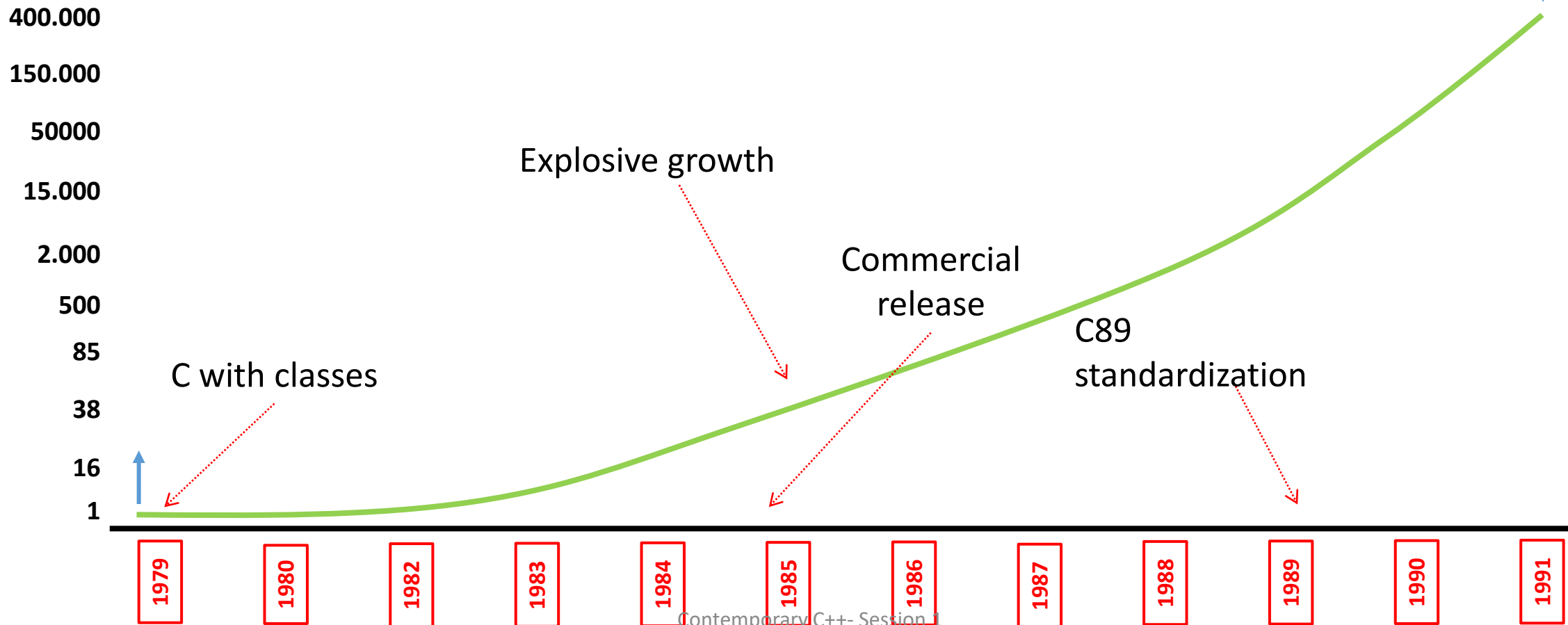
C++ programmers



Is C++ dead?

- 1979-1991
- Main reference: Bjarne Stroustrup. Design and Evolution of C++, 1994.
- During 1980-1991, the number of users doubled every seven and a half months.

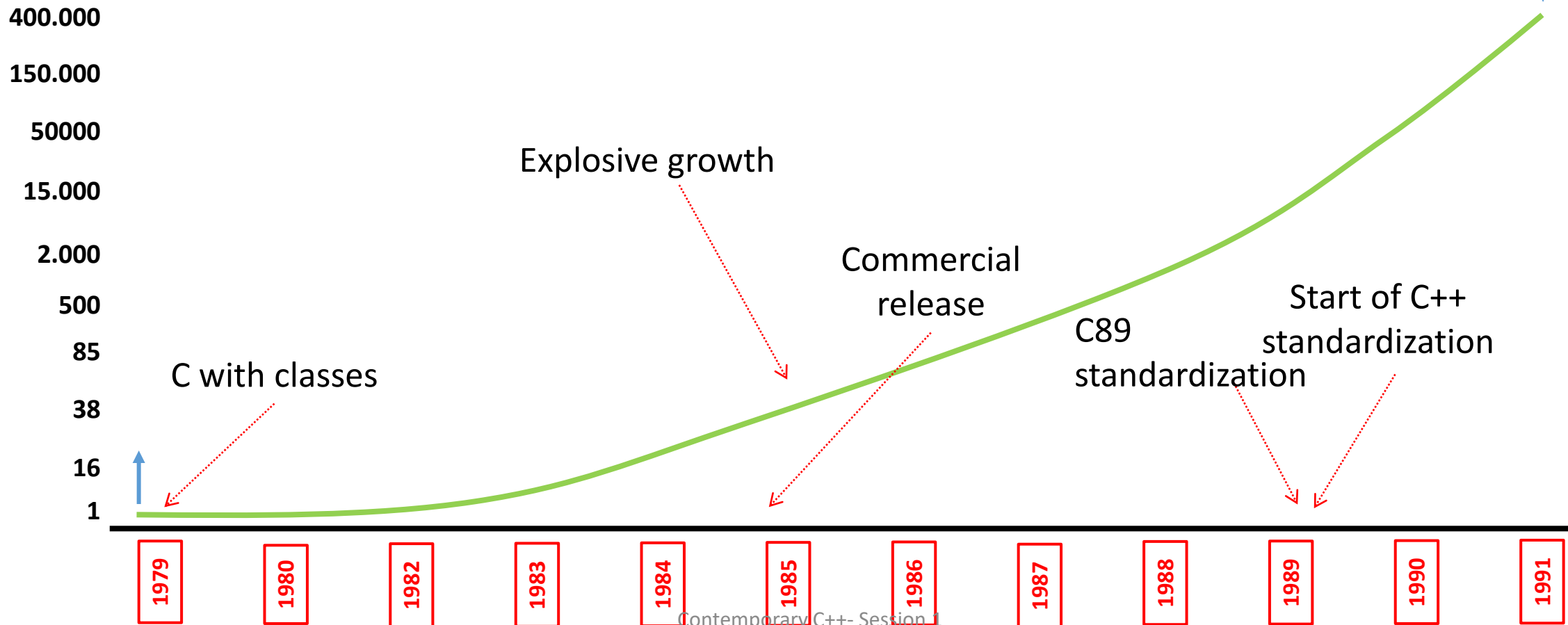
C++ programmers



Is C++ dead?

- 1979-1991
- Main reference: Bjarne Stroustrup. Design and Evolution of C++, 1994.
- During 1980-1991, the number of users doubled every seven and a half months.

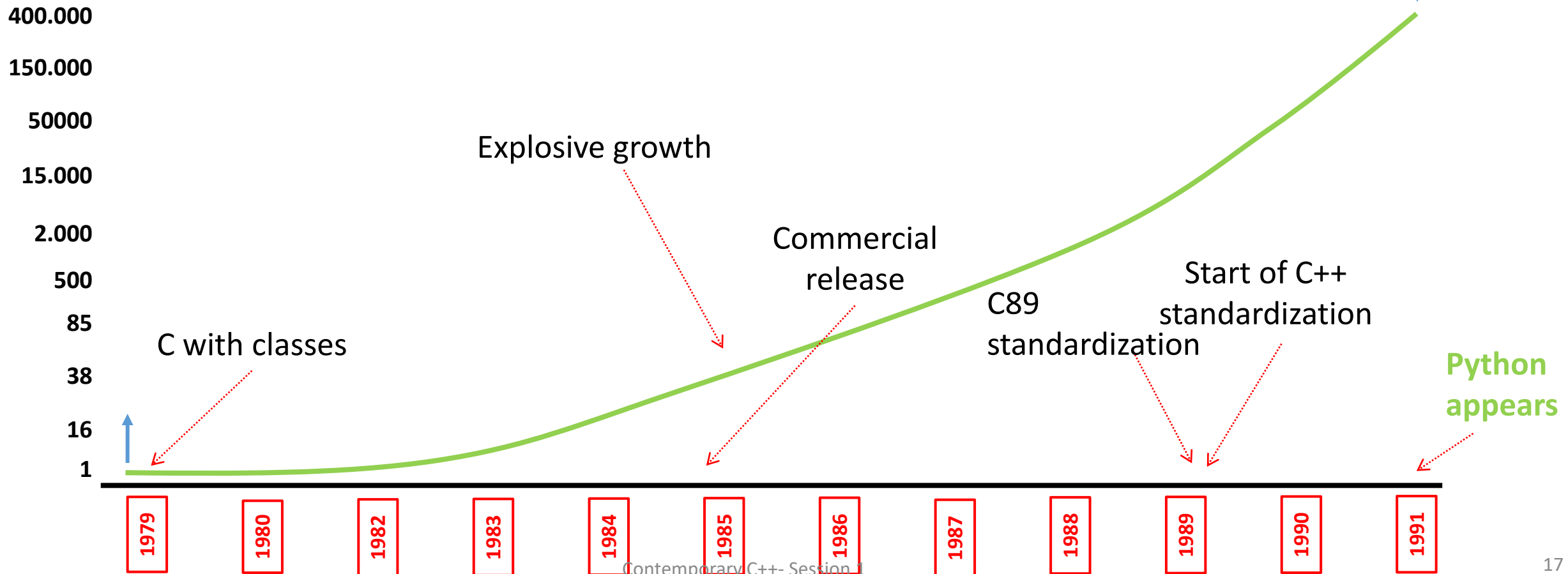
C++ programmers



Is C++ dead?

- 1979-1991
- Main reference: Bjarne Stroustrup. Design and Evolution of C++, 1994.
- During 1980-1991, the number of users doubled every seven and a half months.

C++ programmers

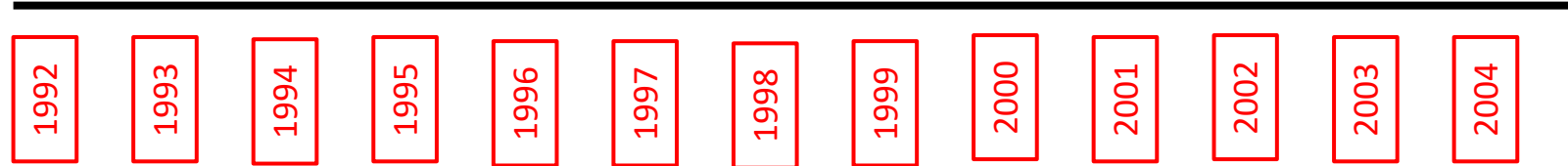


Is C++ dead?

- 1992-2004

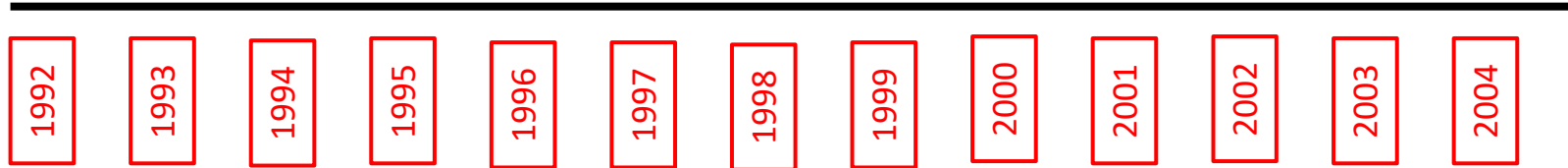
Is C++ dead?

- 1992-2004



Is C++ dead?

- 1992-2004
- Main reference: IDC, Independent Directors Council: www.idc.org

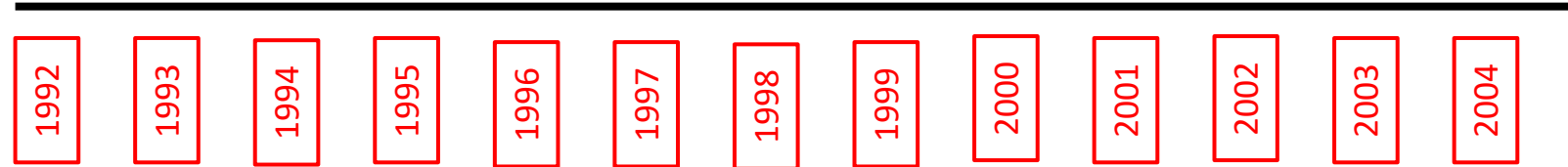


Is C++ dead?

- 1992-2004
- Main reference: IDC, Independent Directors Council: www.idc.org

C++ programmers

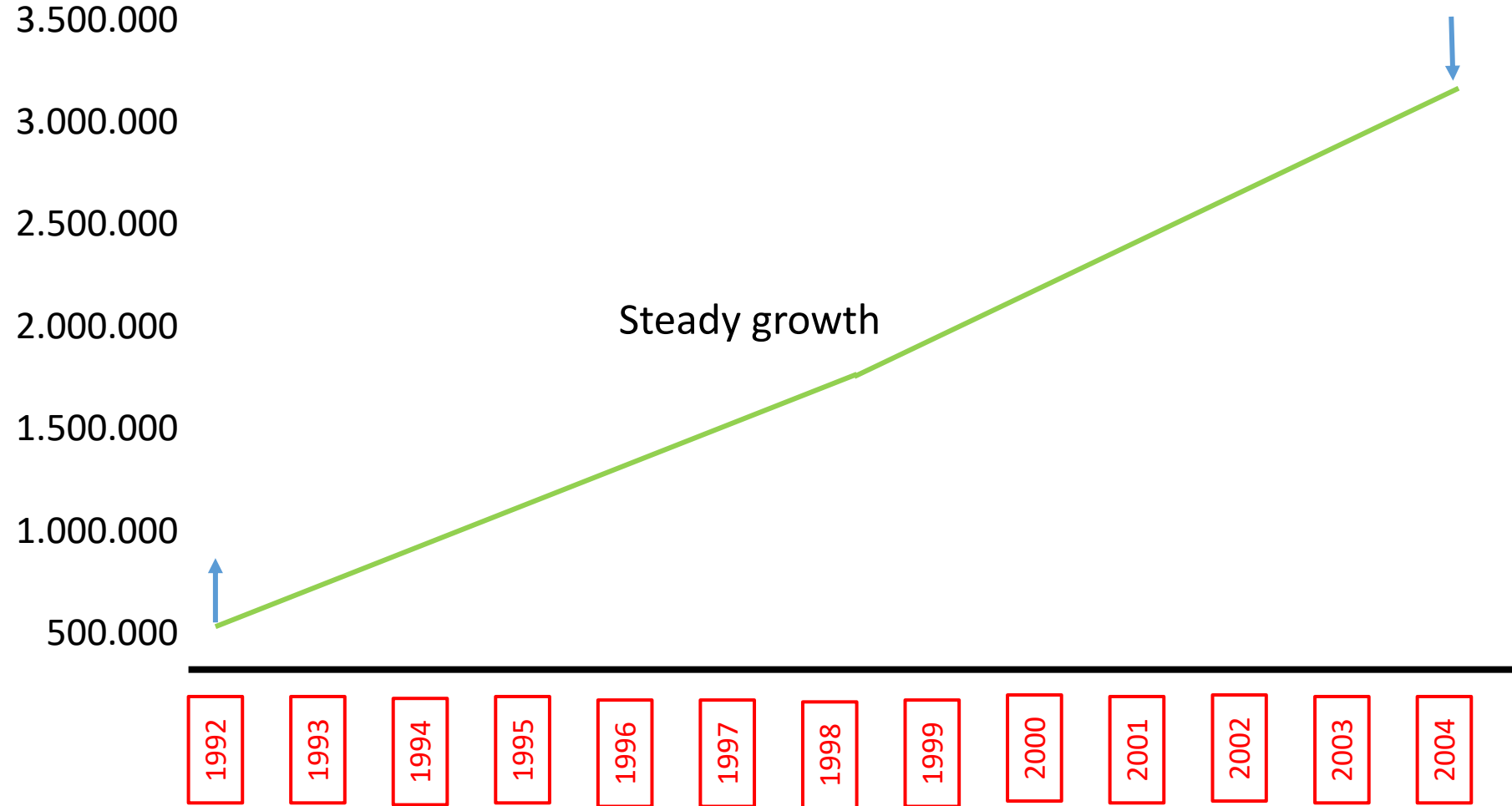
3.500.000
3.000.000
2.500.000
2.000.000
1.500.000
1.000.000
500.000



Is C++ dead?

- 1992-2004
- Main reference: IDC, Independent Directors Council: www.idc.org

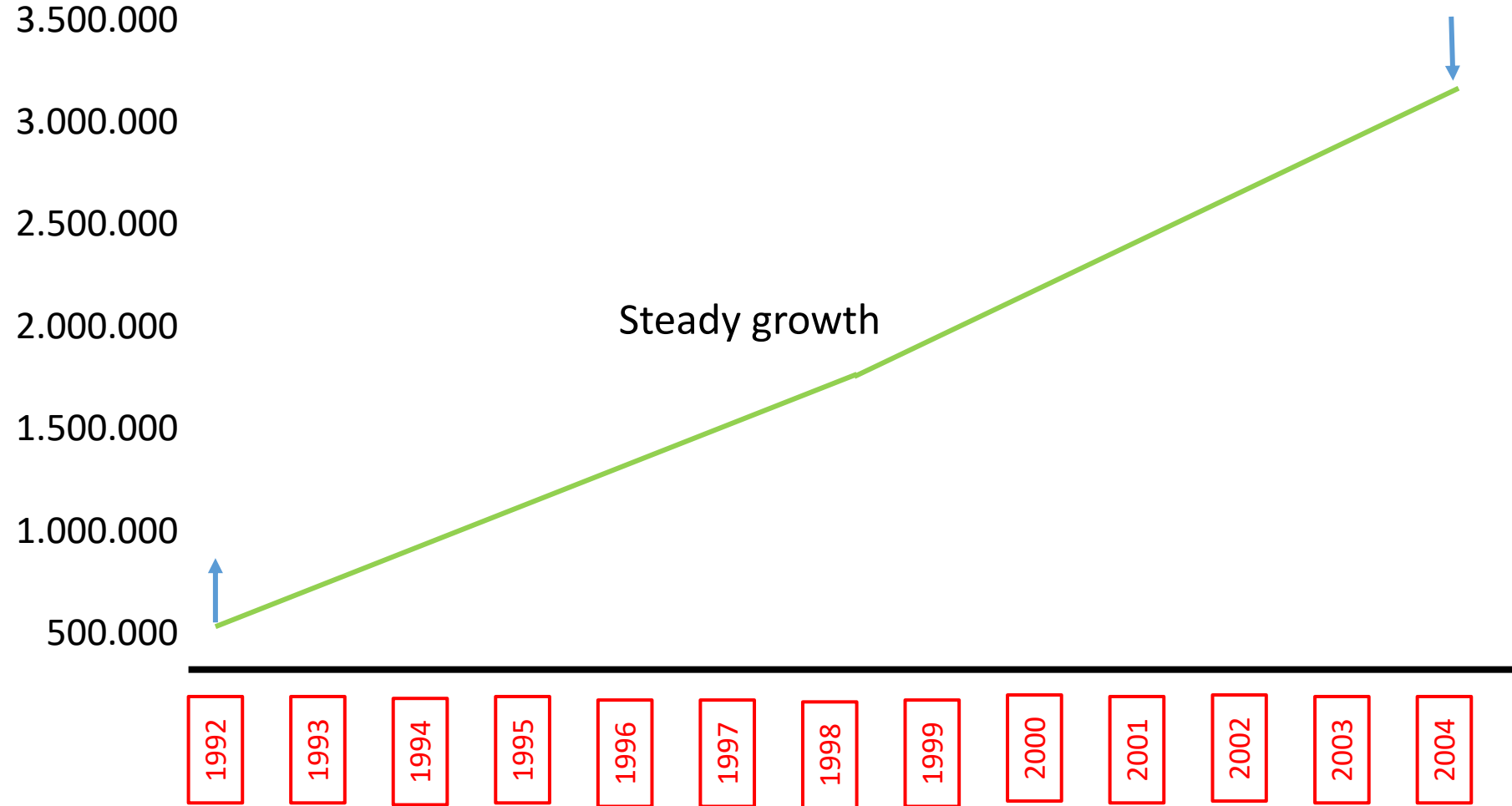
C++ programmers



Is C++ dead?

- 1992-2004
- Main reference: IDC, Independent Directors Council: www.idc.org
- The growth rate: A few percent.

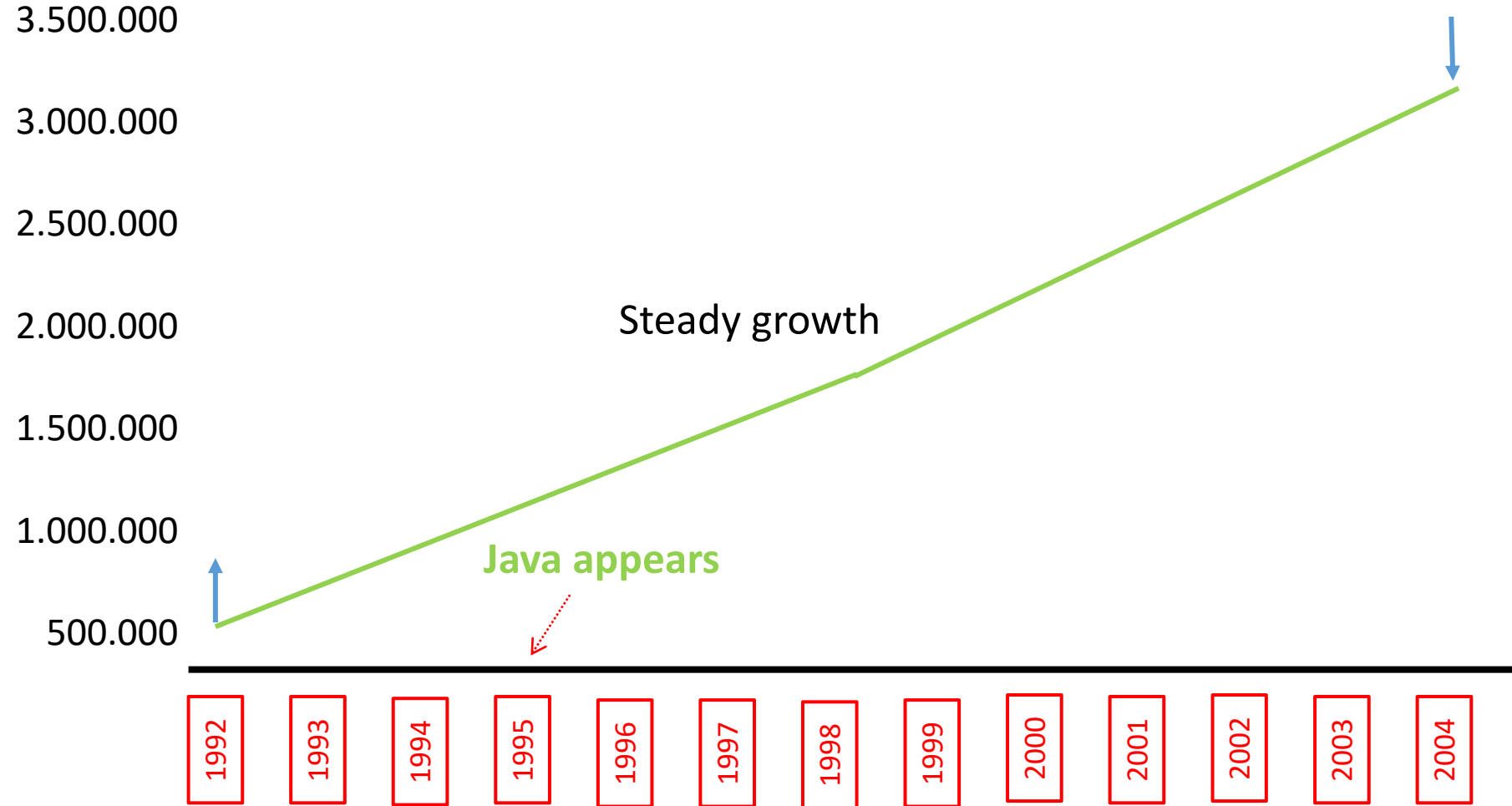
C++ programmers



Is C++ dead?

- 1992-2004
- Main reference: IDC, Independent Directors Council: www.idc.org
- The growth rate: A few percent.

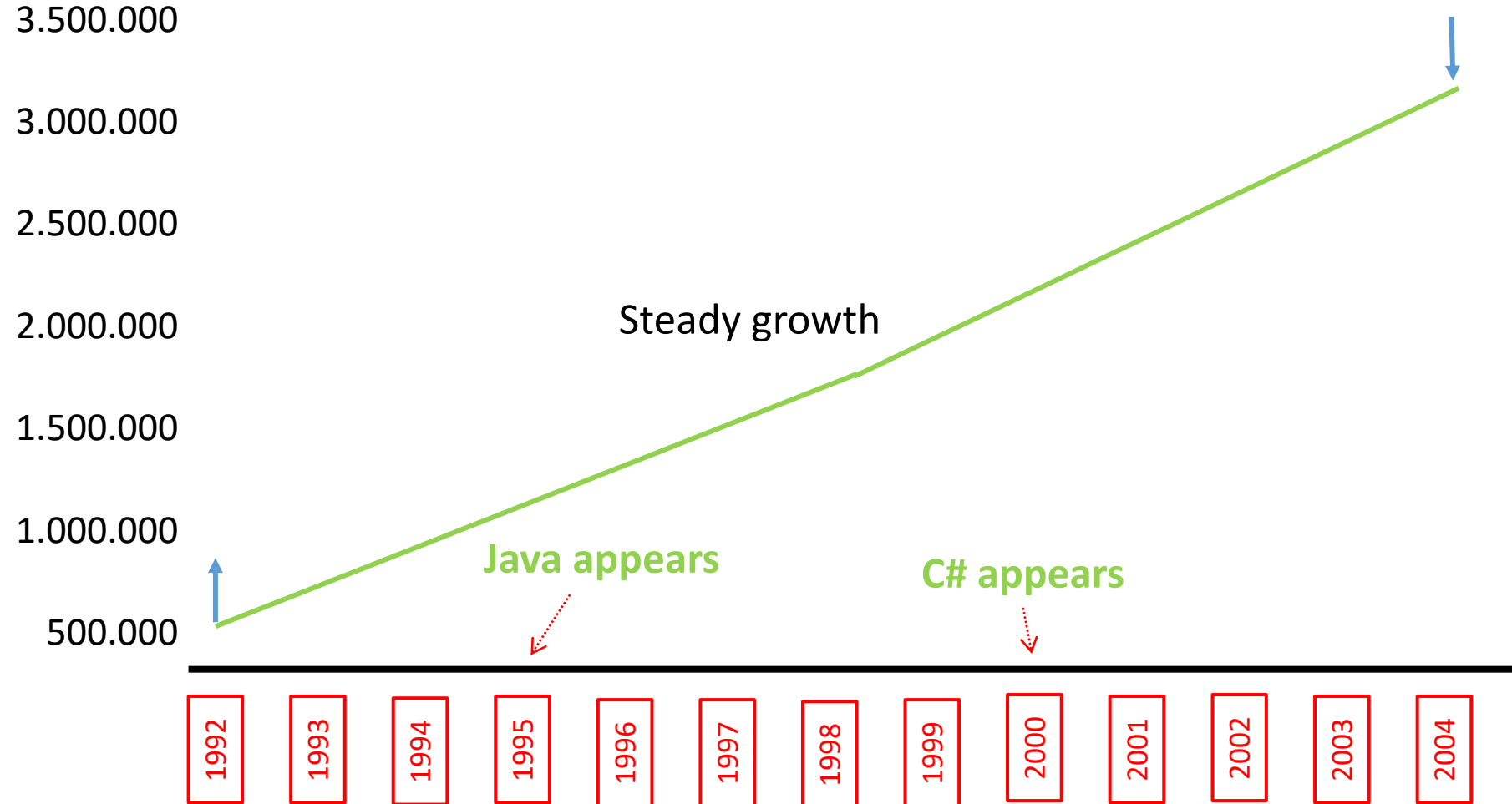
C++ programmers



Is C++ dead?

- 1992-2004
- Main reference: IDC, Independent Directors Council: www.idc.org
- The growth rate: A few percent.

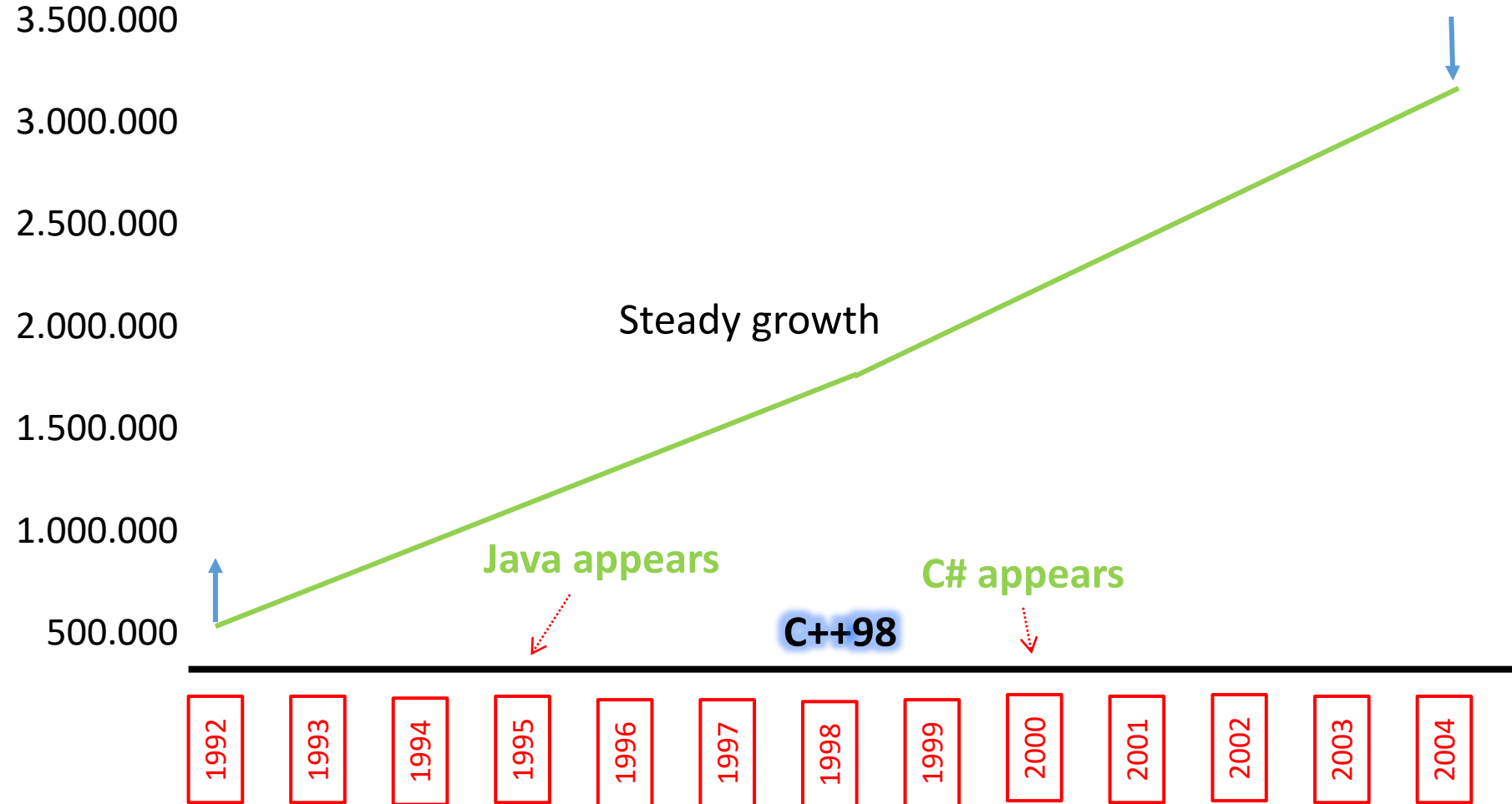
C++ programmers



Is C++ dead?

- 1992-2004
- Main reference: IDC, Independent Directors Council: www.idc.org
- The growth rate: A few percent.

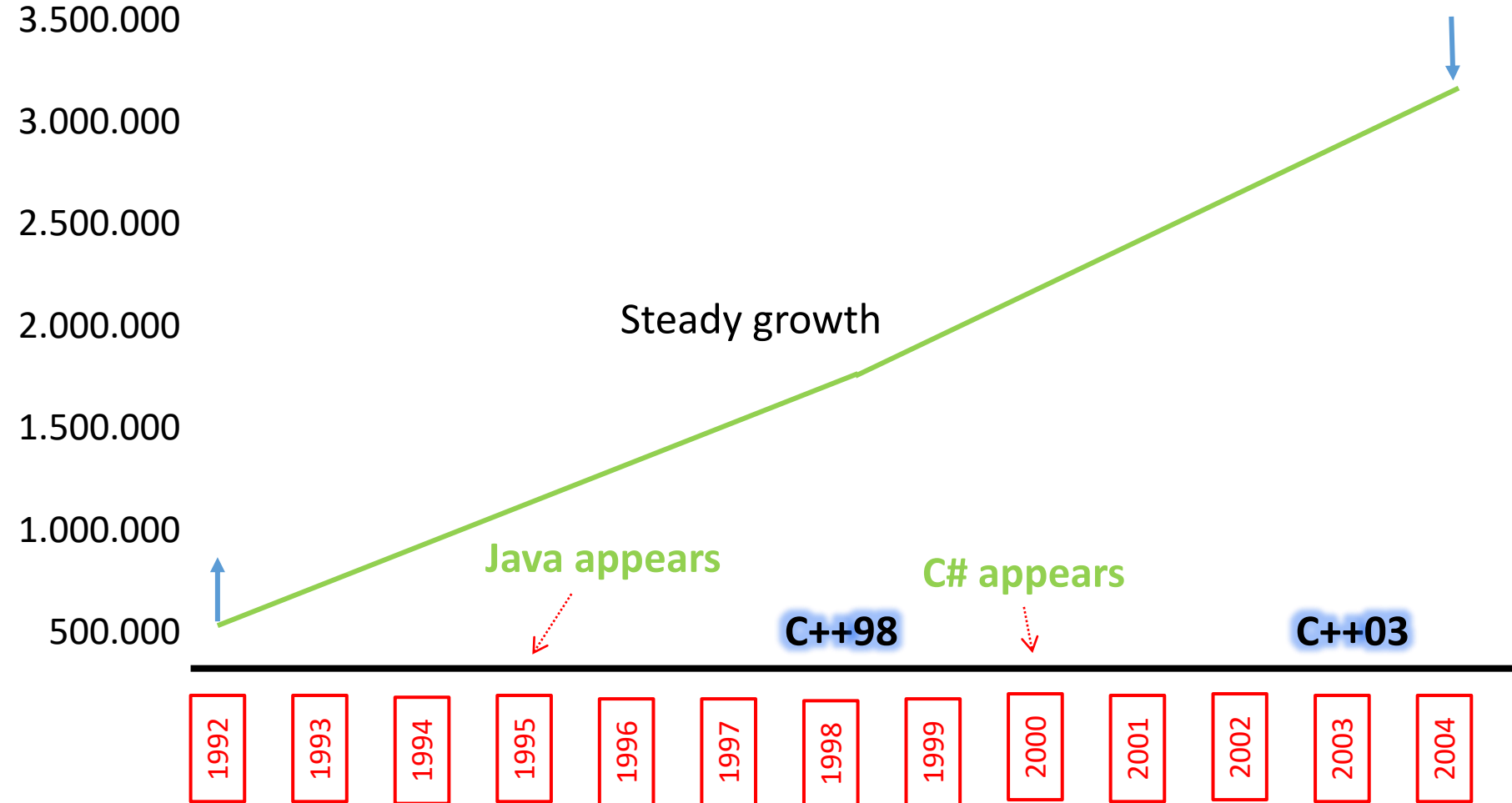
C++ programmers



Is C++ dead?

- 1992-2004
- Main reference: IDC, Independent Directors Council: www.idc.org
- The growth rate: A few percent.

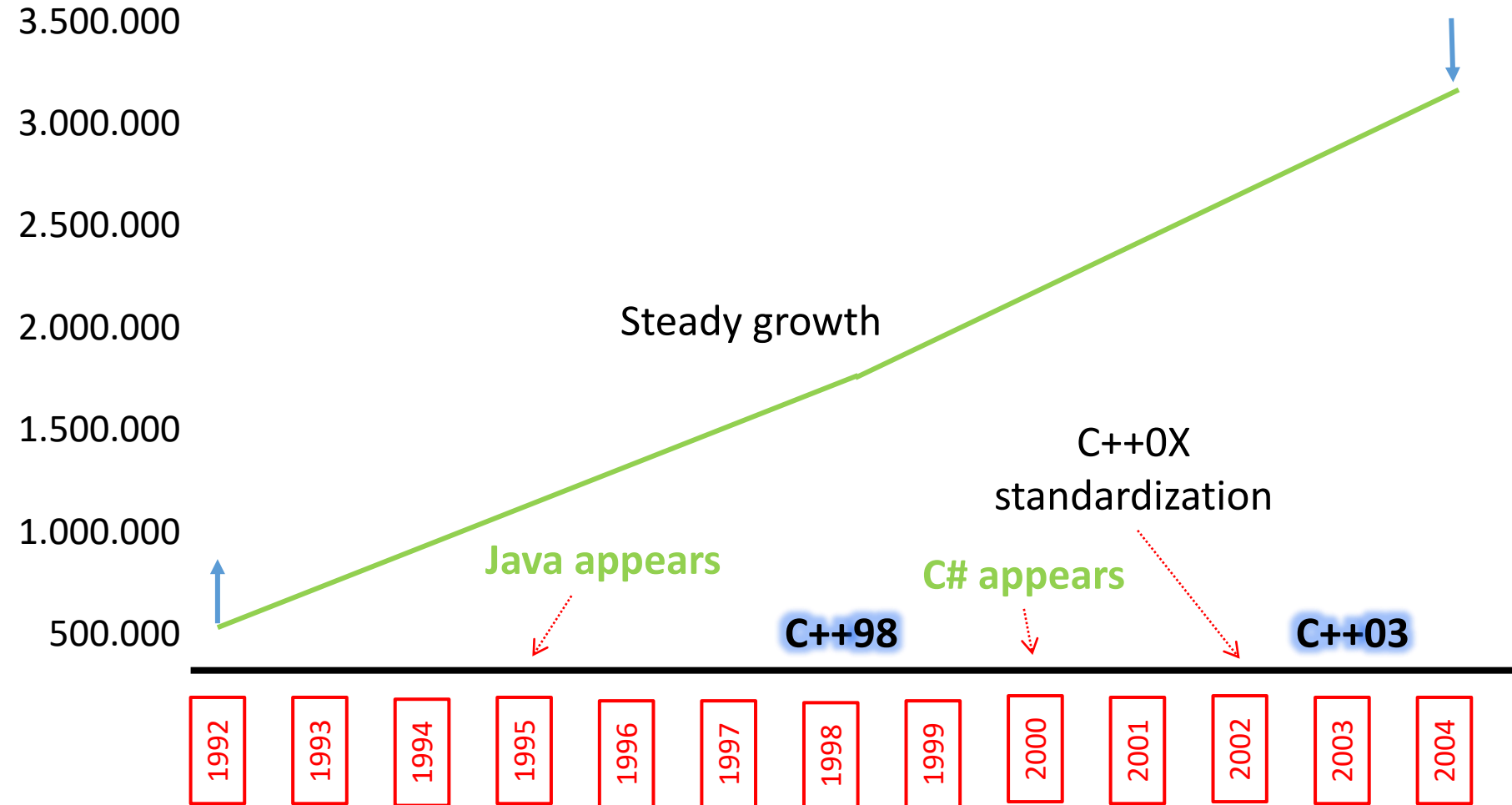
C++ programmers



Is C++ dead?

- 1992-2004
- Main reference: IDC, Independent Directors Council: www.idc.org
- The growth rate: A few percent.

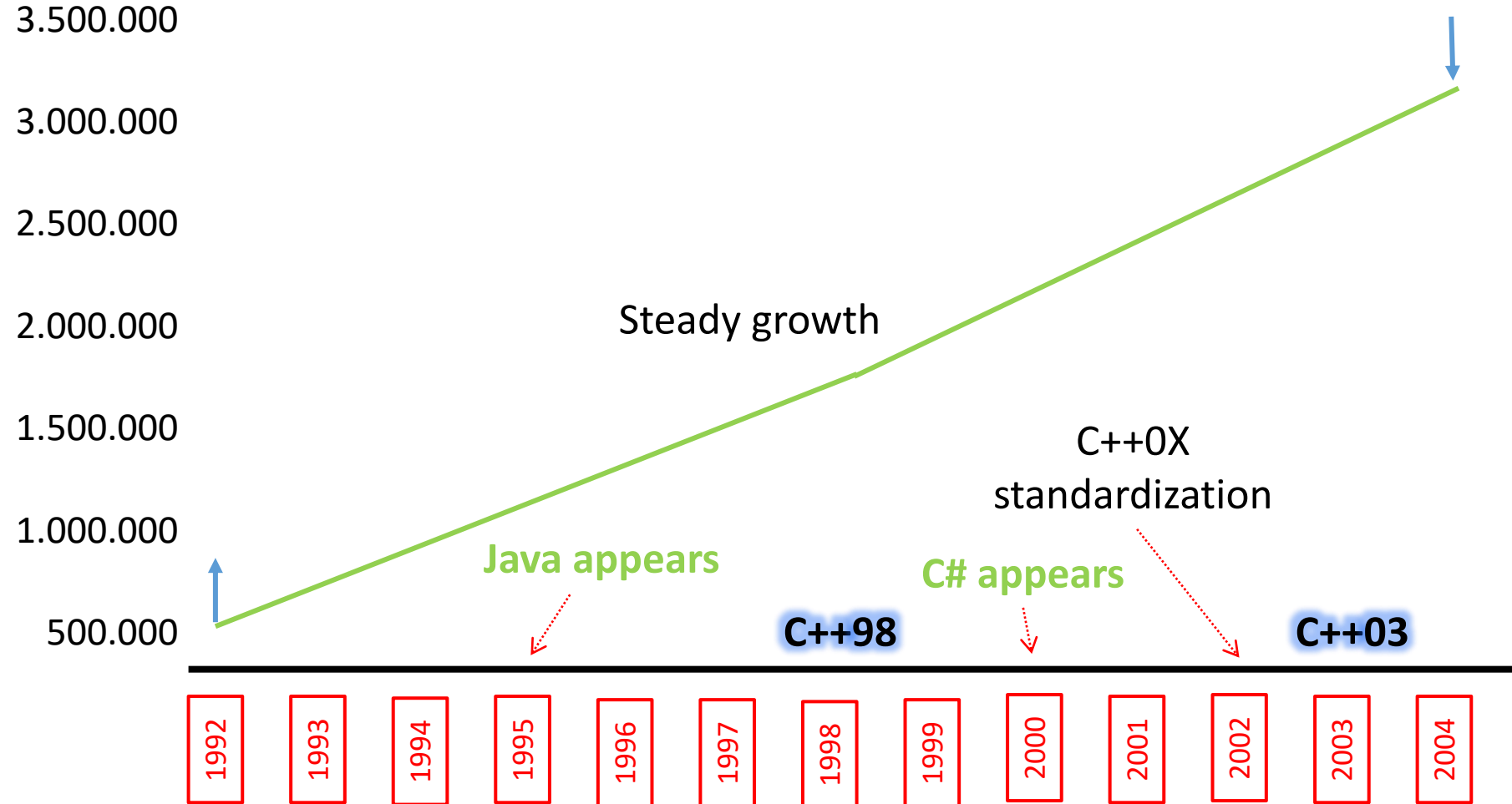
C++ programmers



Is C++ dead?

- 1992-2004
- Main reference: IDC, Independent Directors Council: www.idc.org
- The growth rate: A few percent.

C++ programmers



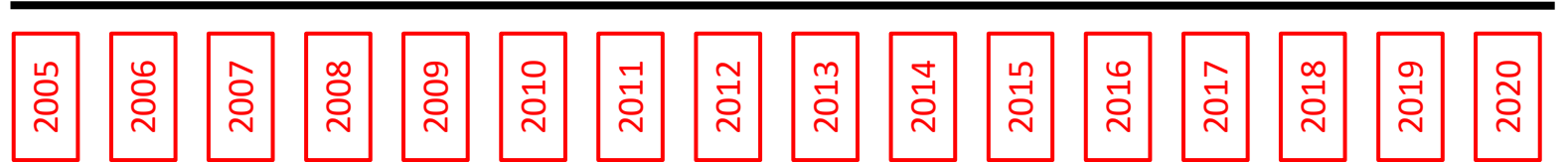
- IDC estimated that at 2001 of the number of C++ programmers was "about 3 million"; their 2004 number was "more than 3 million".

Is C++ dead?

- 2004-2020

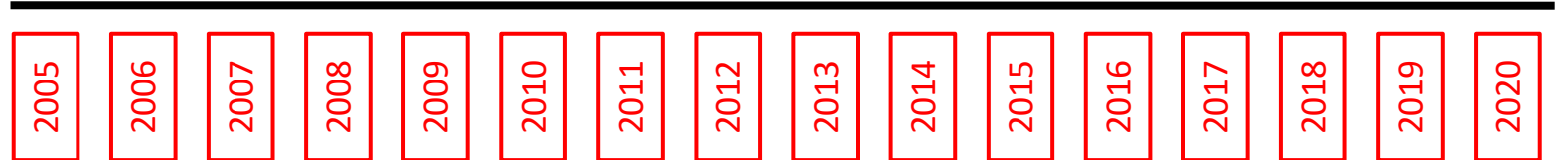
Is C++ dead?

- 2004-2020



Is C++ dead?

- 2004-2020
- Main reference: <https://blog.jetbrains.com/clion/2015/07/infographics-cpp-facts-before-clion/>



Is C++ dead?

- 2004-2020
- Main reference: <https://blog.jetbrains.com/clion/2015/07/infographics-cpp-facts-before-clion/>

C++ programmers

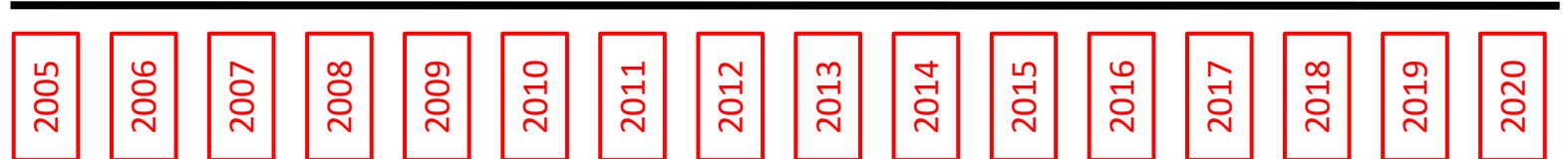
5.000.000

4.500.000

4.000.000

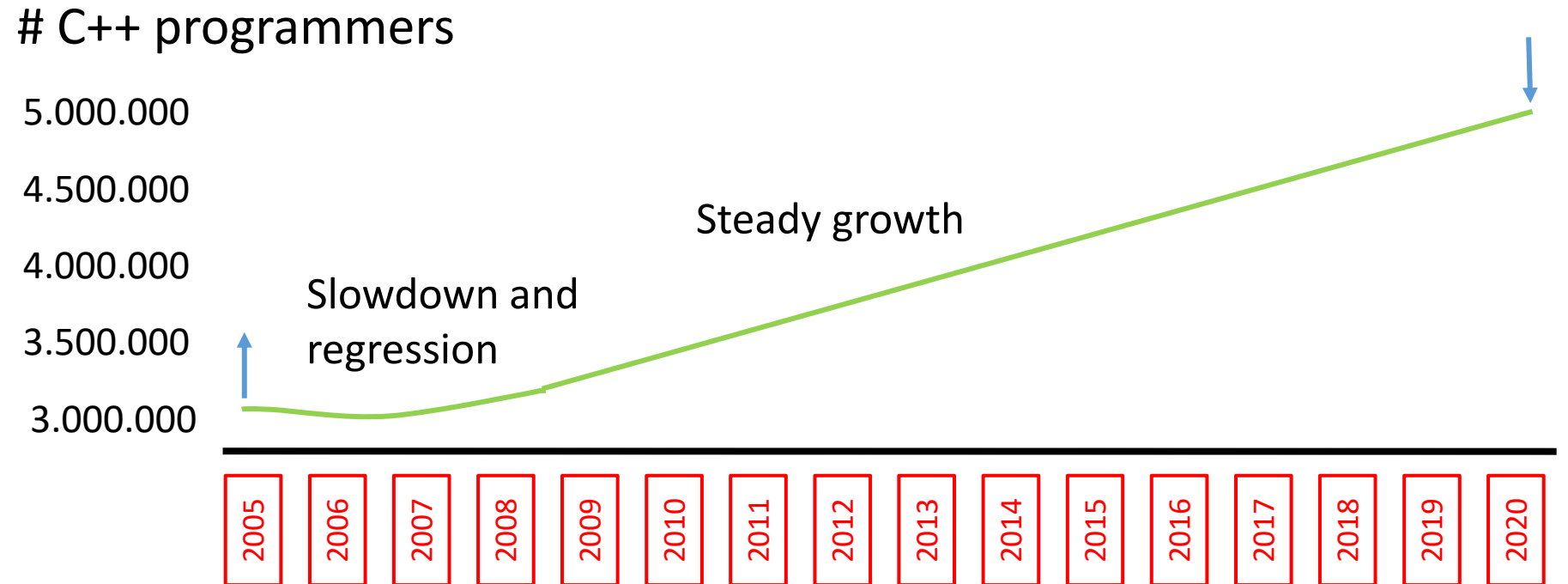
3.500.000

3.000.000



Is C++ dead?

- 2004-2020
- Main reference: <https://blog.jetbrains.com/clion/2015/07/infographics-cpp-facts-before-clion/>



Is C++ dead?

- 2004-2020
- Main reference: <https://blog.jetbrains.com/clion/2015/07/infographics-cpp-facts-before-clion/>
- 2005-2009: Slowdown and regression
- The growth rate: 2-3 precents.

C++ programmers

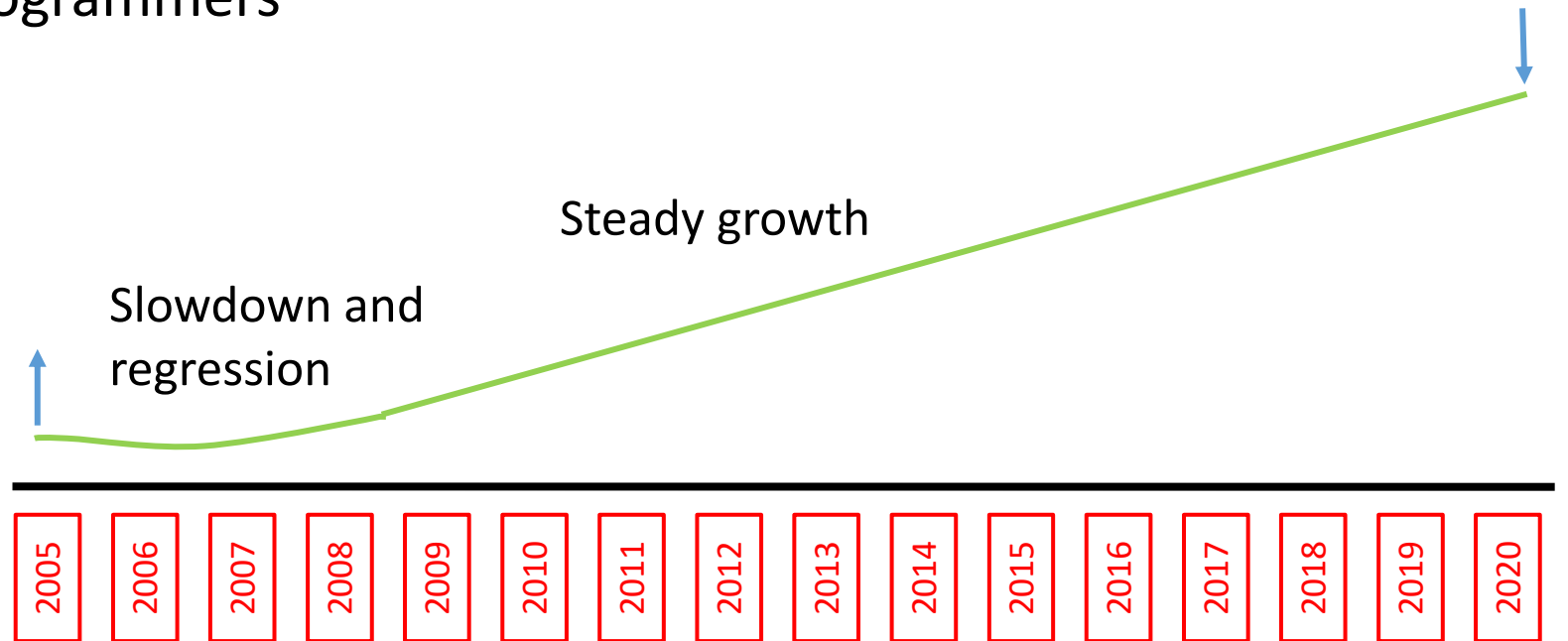
5.000.000

4.500.000

4.000.000

3.500.000

3.000.000



Is C++ dead?

- 2004-2020
- Main reference: <https://blog.jetbrains.com/clion/2015/07/infographics-cpp-facts-before-clion/>
- 2005-2009: Slowdown and regression
- The growth rate: 2-3 precents.

C++ programmers

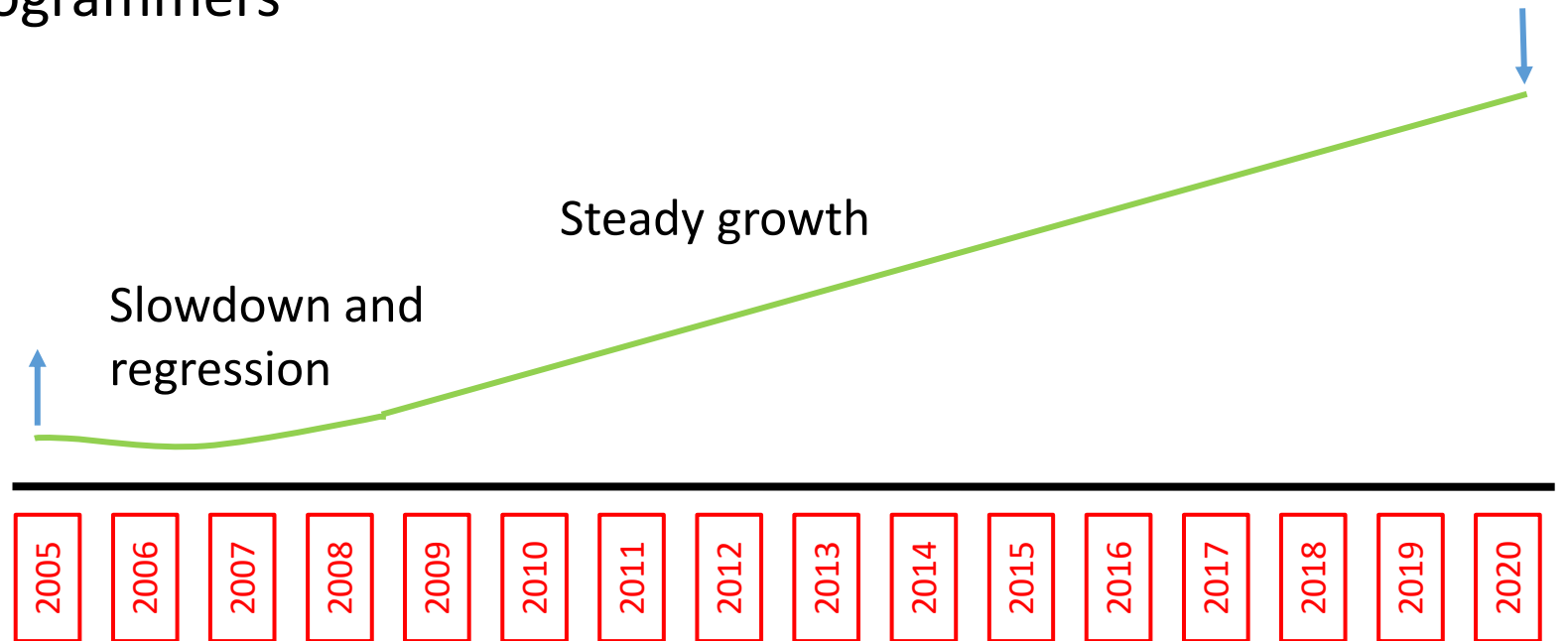
5.000.000

4.500.000

4.000.000

3.500.000

3.000.000



Go appears

Is C++ dead?

- 2004-2020
- Main reference: <https://blog.jetbrains.com/clion/2015/07/infographics-cpp-facts-before-clion/>
- 2005-2009: Slowdown and regression
- The growth rate: 2-3 precents.

C++ programmers

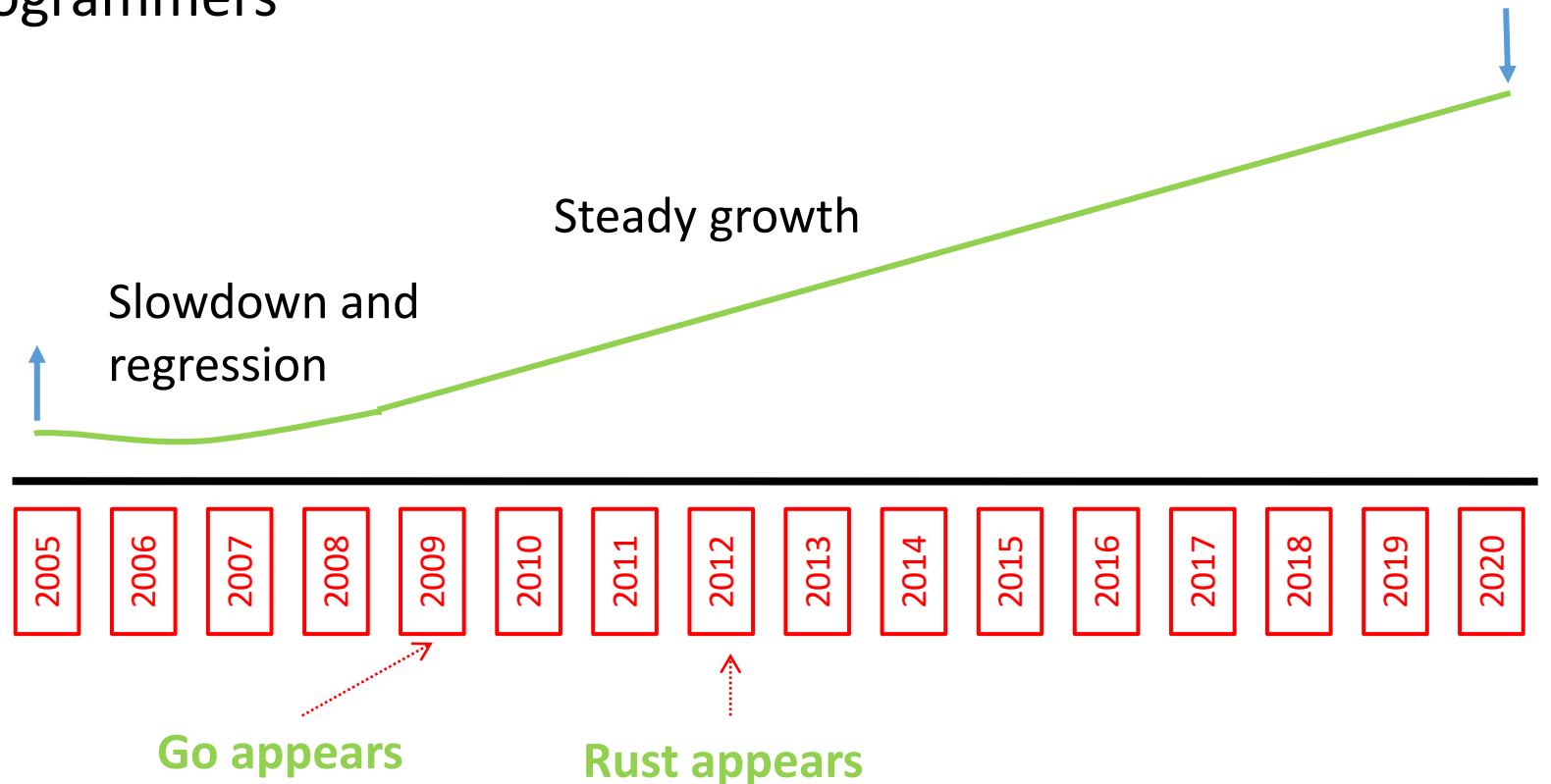
5.000.000

4.500.000

4.000.000

3.500.000

3.000.000



Is C++ dead?

- 2004-2020
- Main reference: <https://blog.jetbrains.com/clion/2015/07/infographics-cpp-facts-before-clion/>
- 2005-2009: Slowdown and regression
- The growth rate: 2-3 precents.

C++ programmers

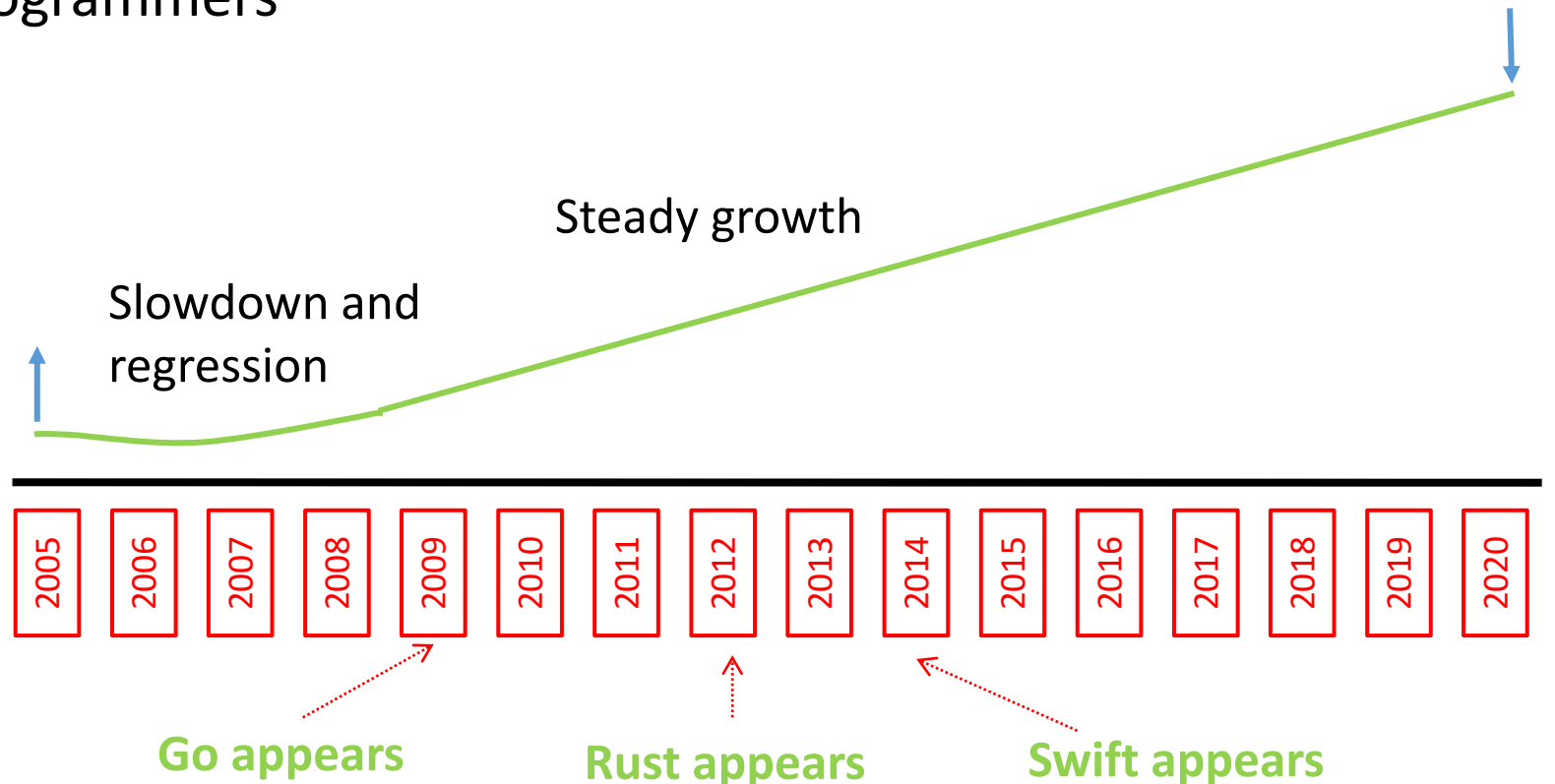
5.000.000

4.500.000

4.000.000

3.500.000

3.000.000



Is C++ dead?

- 2004-2020
- Main reference: <https://blog.jetbrains.com/clion/2015/07/infographics-cpp-facts-before-clion/>
- 2005-2009: Slowdown and regression
- The growth rate: 2-3 precents.

C++ programmers

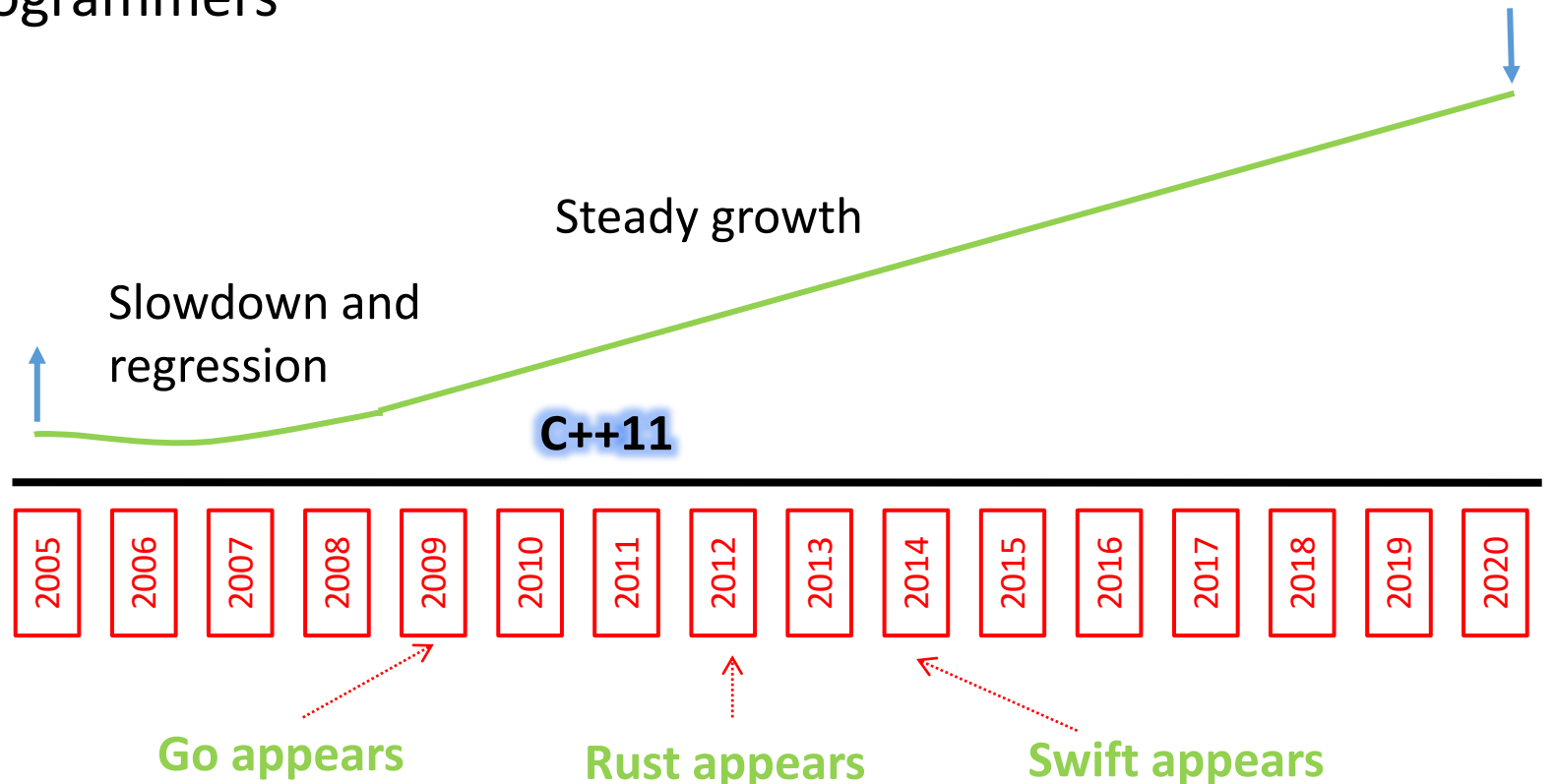
5.000.000

4.500.000

4.000.000

3.500.000

3.000.000



Is C++ dead?

- 2004-2020
- Main reference: <https://blog.jetbrains.com/clion/2015/07/infographics-cpp-facts-before-clion/>
- 2005-2009: Slowdown and regression
- The growth rate: 2-3 precents.

C++ programmers

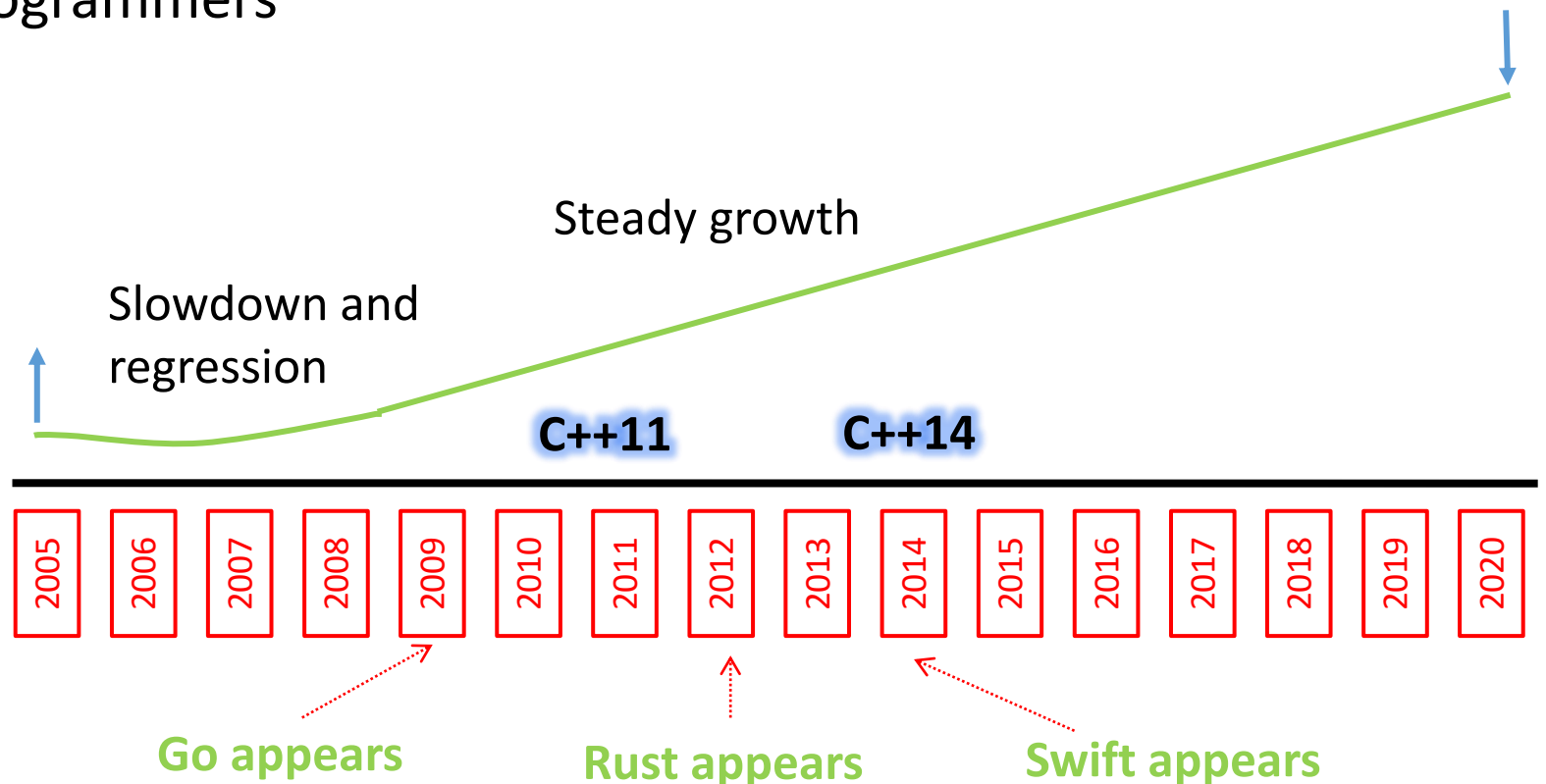
5.000.000

4.500.000

4.000.000

3.500.000

3.000.000



Is C++ dead?

- 2004-2020
- Main reference: <https://blog.jetbrains.com/clion/2015/07/infographics-cpp-facts-before-clion/>
- 2005-2009: Slowdown and regression
- The growth rate: 2-3 precents.

C++ programmers

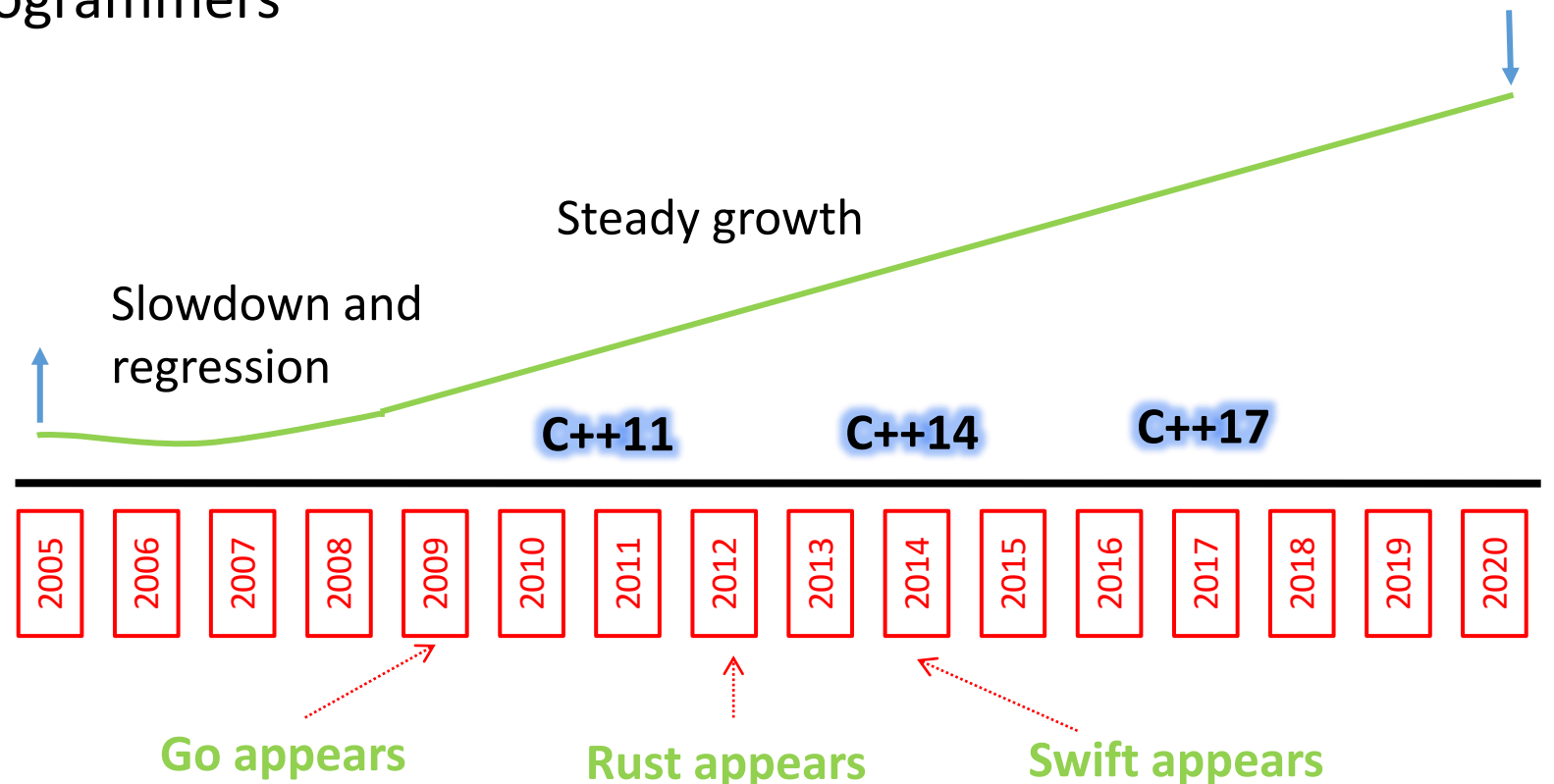
5.000.000

4.500.000

4.000.000

3.500.000

3.000.000



Is C++ dead?

- 2004-2020
- Main reference: <https://blog.jetbrains.com/clion/2015/07/infographics-cpp-facts-before-clion/>
- 2005-2009: Slowdown and regression
- The growth rate: 2-3 precents.

C++ programmers

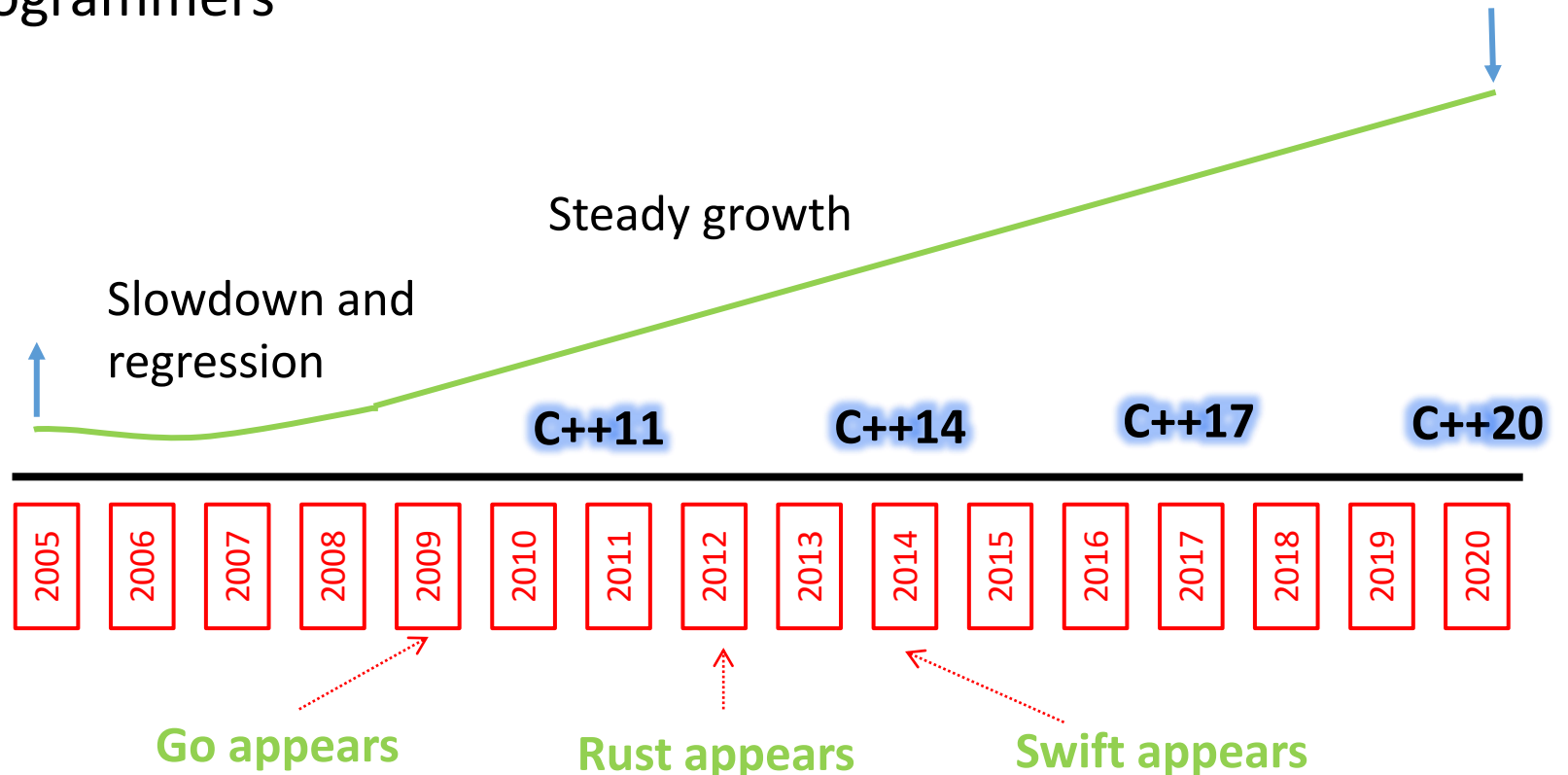
5.000.000

4.500.000

4.000.000

3.500.000

3.000.000



Is C++ dead?

- 2004-2020
- Main reference: <https://blog.jetbrains.com/clion/2015/07/infographics-cpp-facts-before-clion/>
- 2005-2009: Slowdown and regression
- The growth rate: 2-3 precents.

C++ programmers

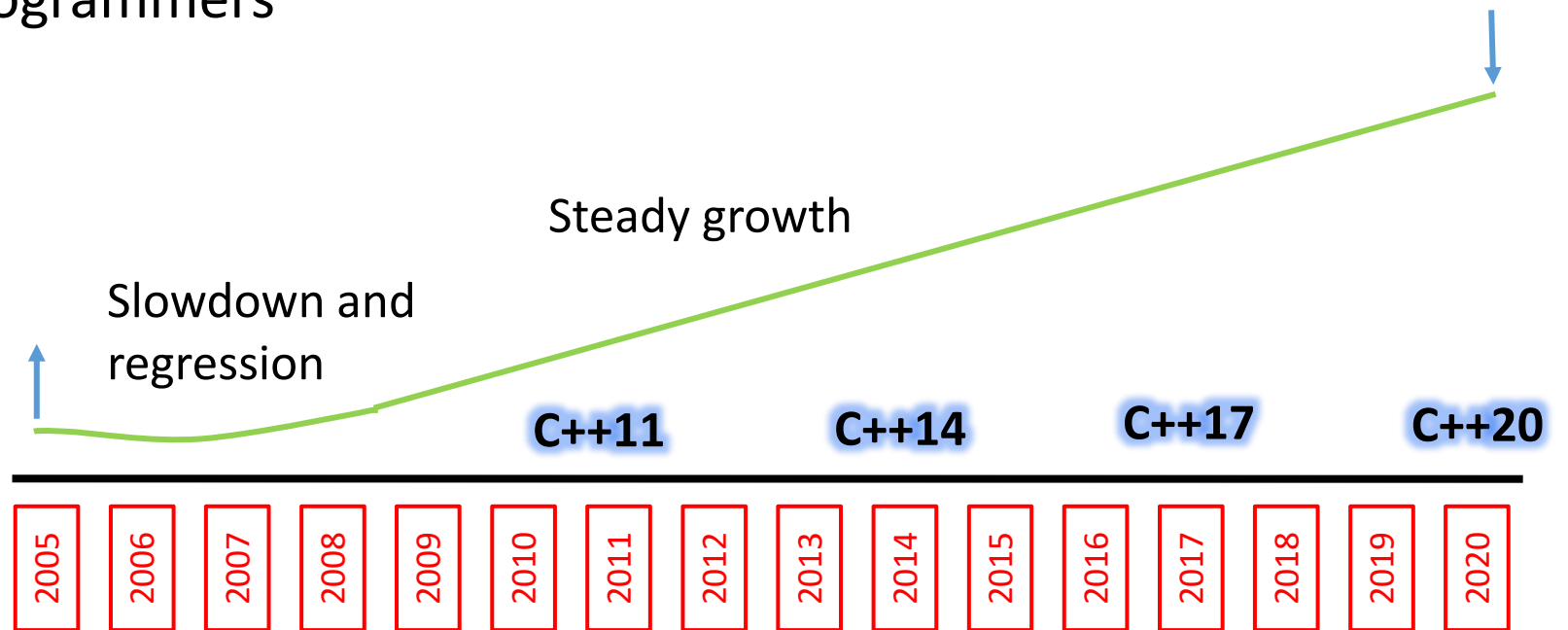
5.000.000

4.500.000

4.000.000

3.500.000

3.000.000



- At 2015: ~ 4.4 million C++ developers
~ 1.9 million C developers

Go appears

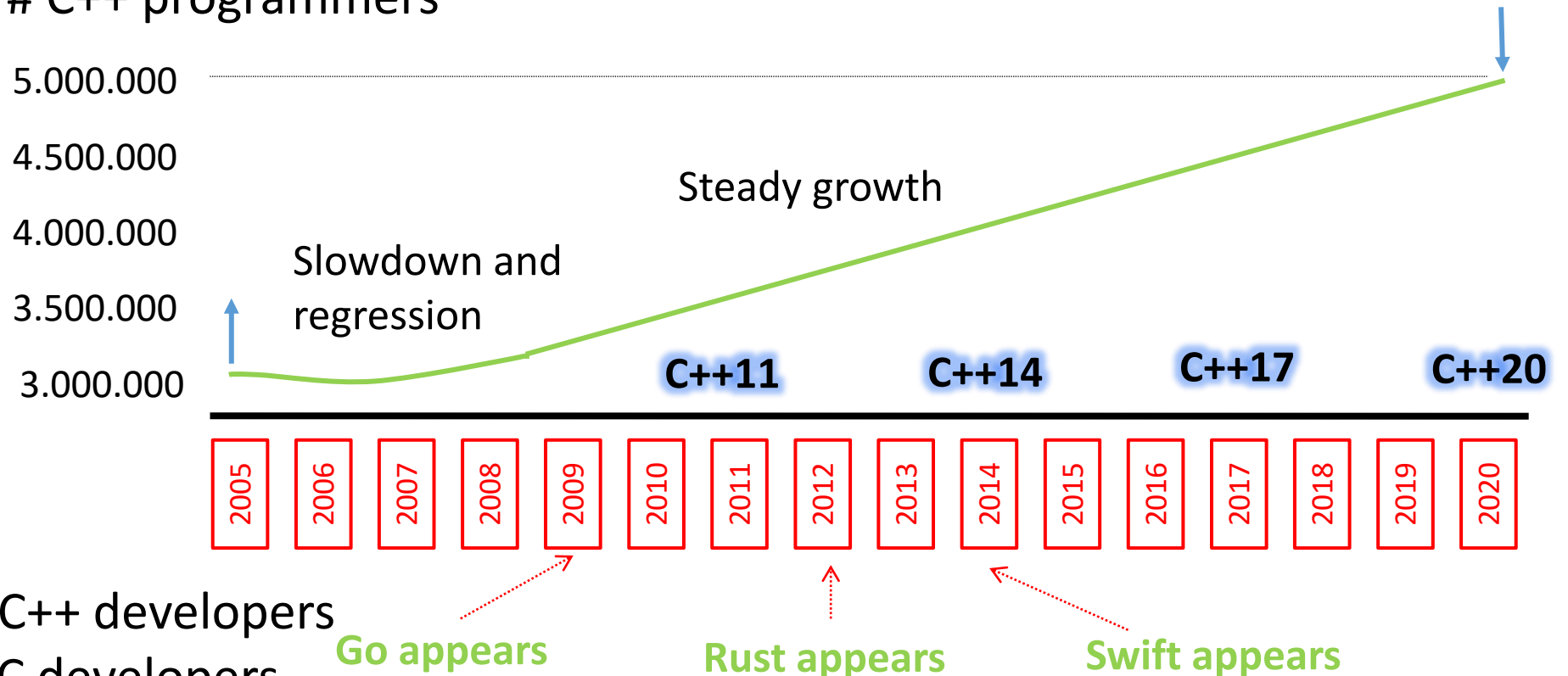
Rust appears

Swift appears

Is C++ dead?

- 2004-2020
- Main reference: <https://blog.jetbrains.com/clion/2015/07/infographics-cpp-facts-before-clion/>
- 2005-2009: Slowdown and regression
- The growth rate: 2-3 precents.

C++ programmers



- At 2015: ~ 4.4 million C++ developers
~ 1.9 million C developers

Four giant C++ applications

C++ inside

C++ inside Higgs boson particles

C++ *inside* Higgs boson particles



- The European Organization for Nuclear Research known as CERN

C++ *inside* Higgs boson particles



- The European Organization for Nuclear Research known as CERN
- CERN operates the largest particle physics laboratory in the world.

C++ *inside* Higgs boson particles



- The European Organization for Nuclear Research known as CERN
- CERN operates the largest particle physics laboratory in the world.

- CERN (Large Hadron Collider)

C++ *inside* Higgs boson particles



- The European Organization for Nuclear Research known as CERN
- CERN operates the largest particle physics laboratory in the world.

- CERN (Large Hadron Collider)

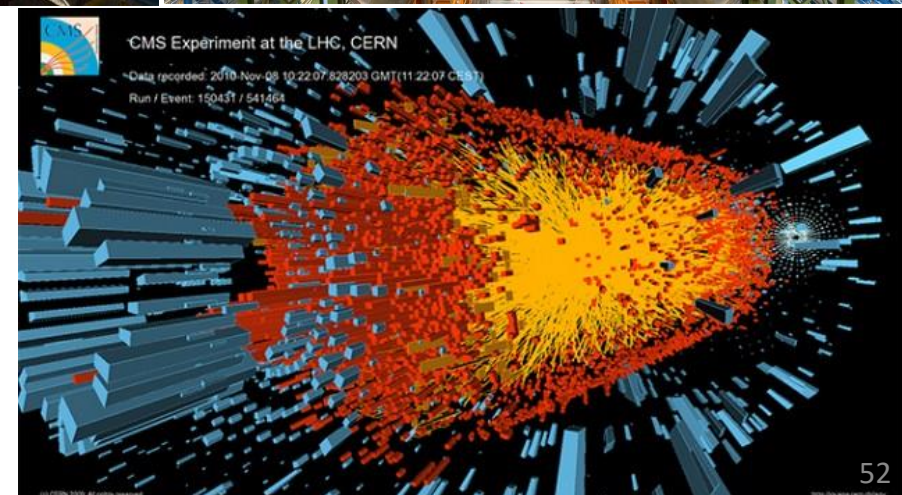
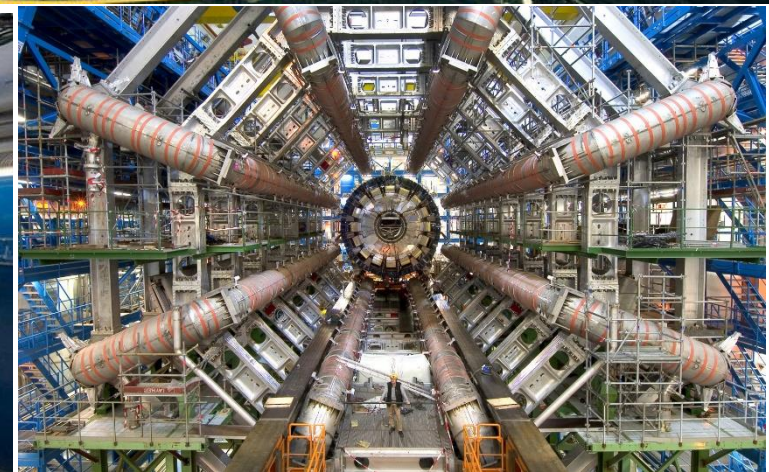
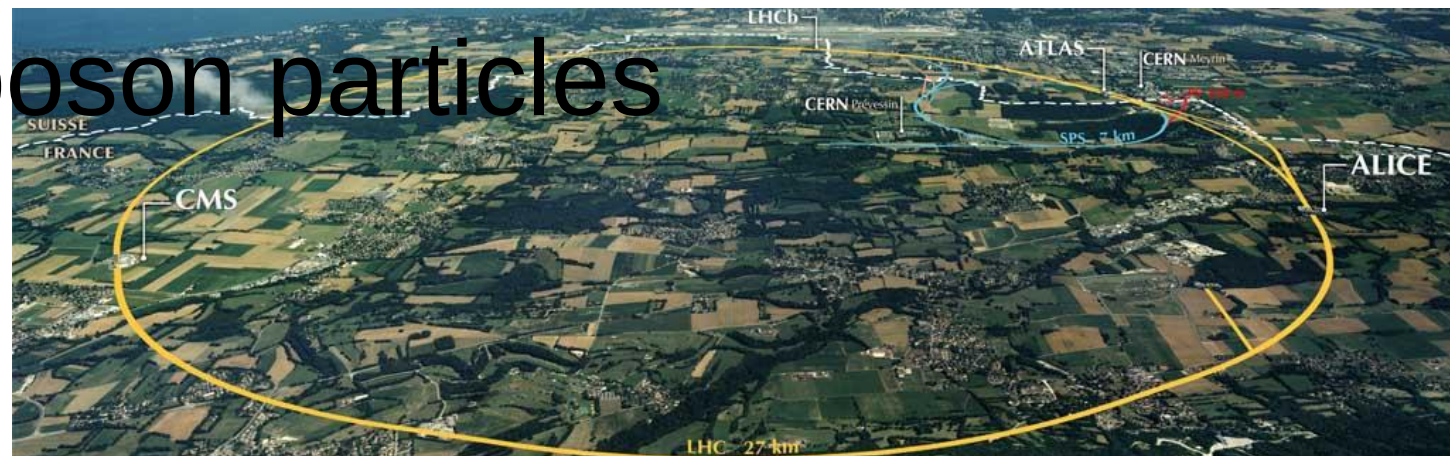


C++ *inside* Higgs boson particles



- The European Organization for Nuclear Research known as CERN
- CERN operates the largest particle physics laboratory in the world.

- CERN (Large Hadron Collider)



C++ *inside* Higgs boson particles



- The European Organization for Nuclear Research known as CERN
- CERN operates the largest particle physics laboratory in the world.

- CERN (Large Hadron Collider)
- -At CERN- all related computing done in C++.

Axel Neumann

[c++std-core-21966] Higgs boson

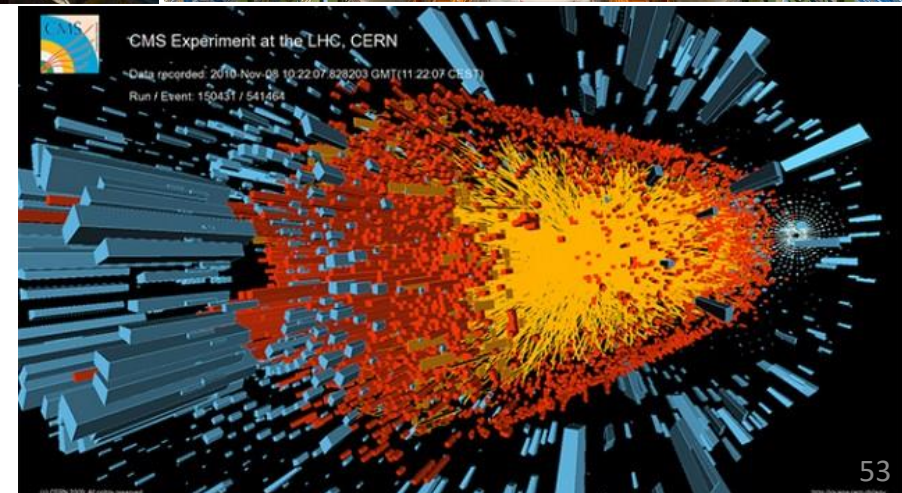
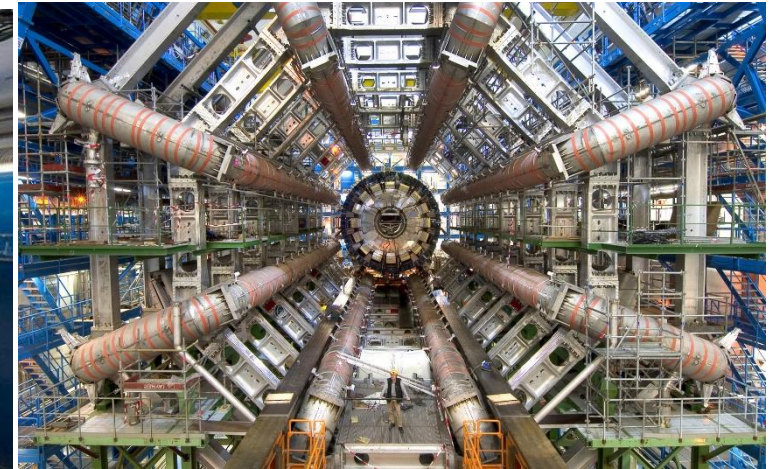
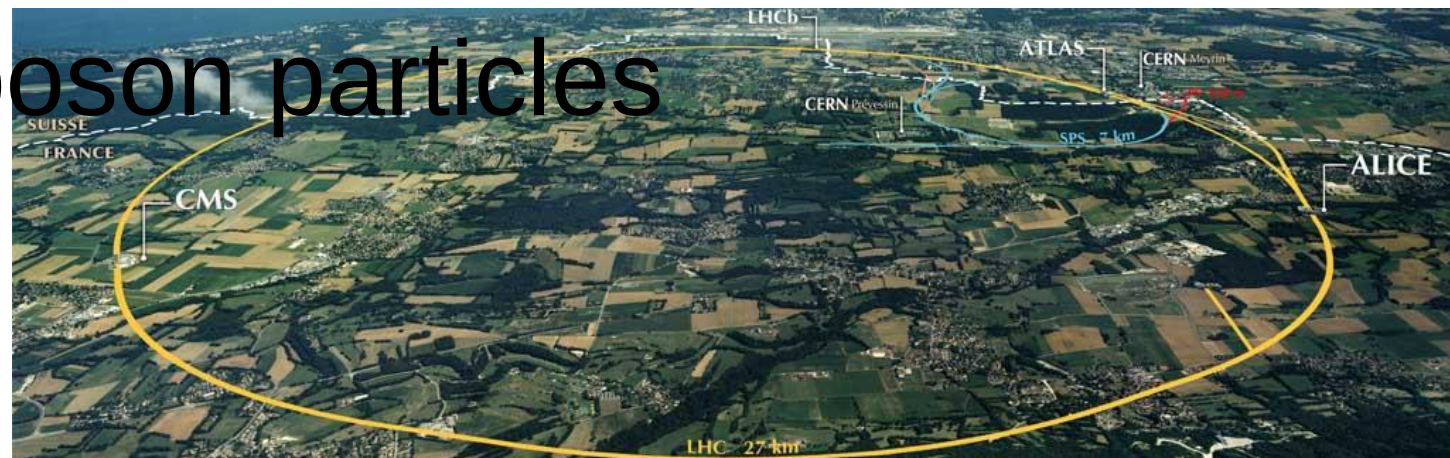
Bjarne Stroustrup

Wed 7/4/2012, 5:08 PM

c++std-news@accu.org (c++std-core@accu.org)

From a short note I received from CERN this morning: "all related computing done in C++."

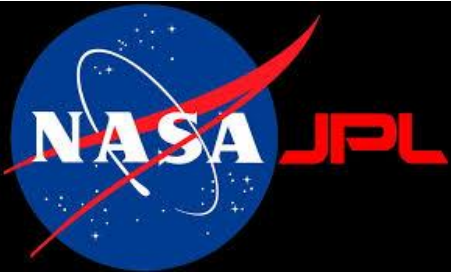
What we do matter.



C++ inside

C++ inside

Mars Rover



- The Jet Propulsion Laboratory (JPL)
- The JPL is owned by NASA and managed by the nearby California Institute of Technology (Caltech) for NASA.

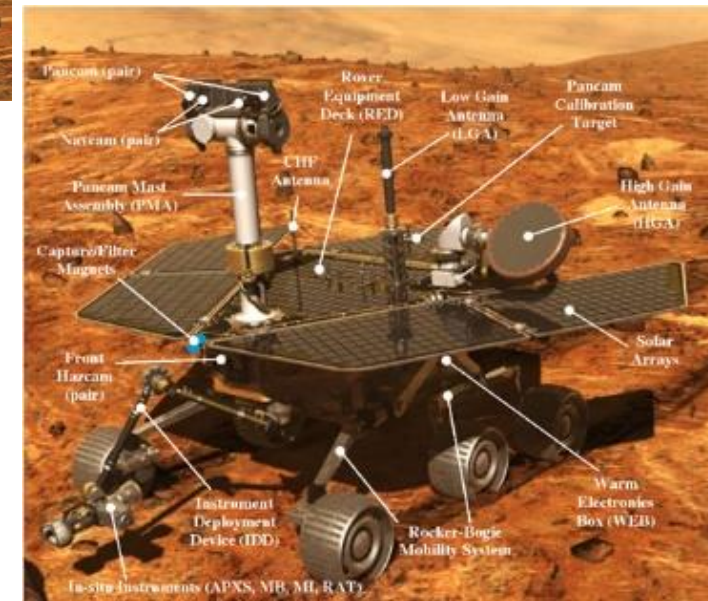
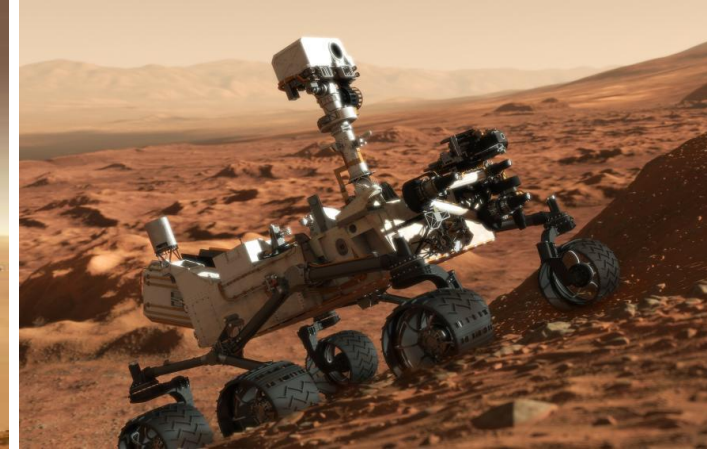
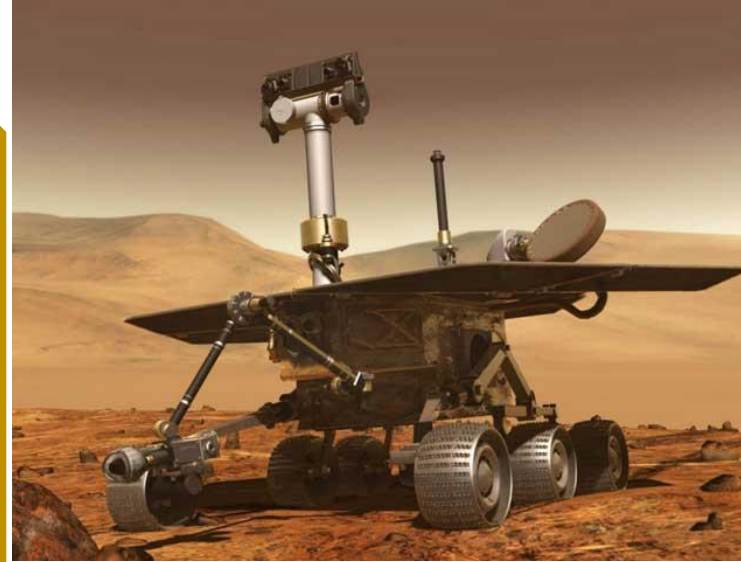
- <https://www.jpl.nasa.gov/>

Mars Rover



- The Jet Propulsion Laboratory (JPL)
- The JPL is owned by NASA and managed by the nearby California Institute of Technology (Caltech) for NASA.

• <https://www.jpl.nasa.gov/>

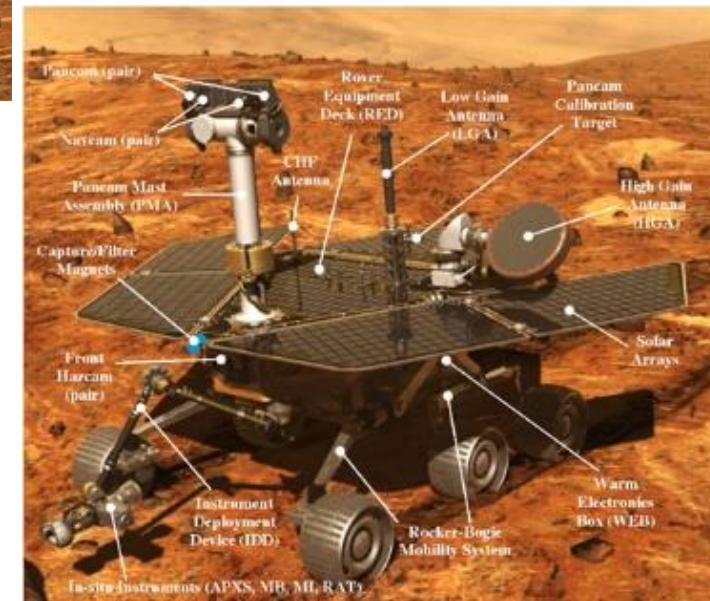
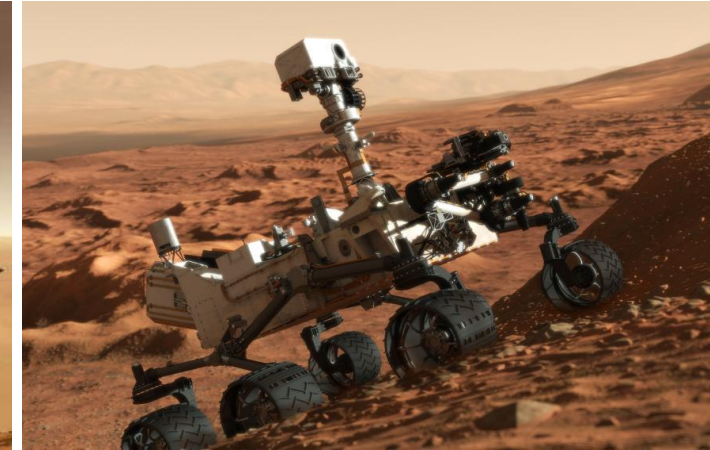
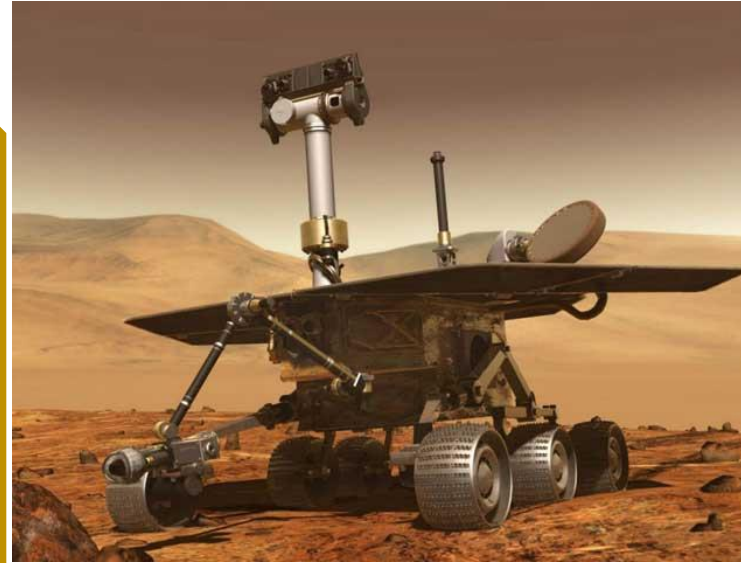


Mars Rover



- The Jet Propulsion Laboratory (JPL)
- The JPL is owned by NASA and managed by the nearby California Institute of Technology (Caltech) for NASA.

• <https://www.jpl.nasa.gov/>



- Mars Rover autonomous driving system: Scene analysis & Route planning
- Spirit, Curiosity, Opportunity, and Perseverance

C++ inside

C++ inside **Human Genome Sequencing Project**

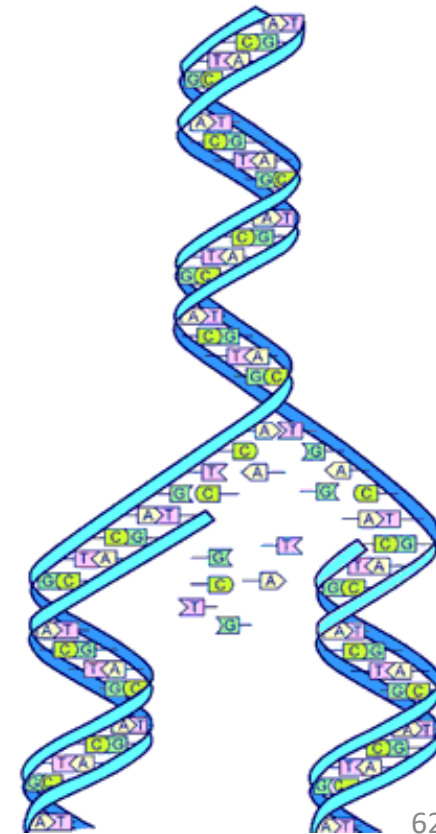
C++ inside Human Genome Sequencing Project



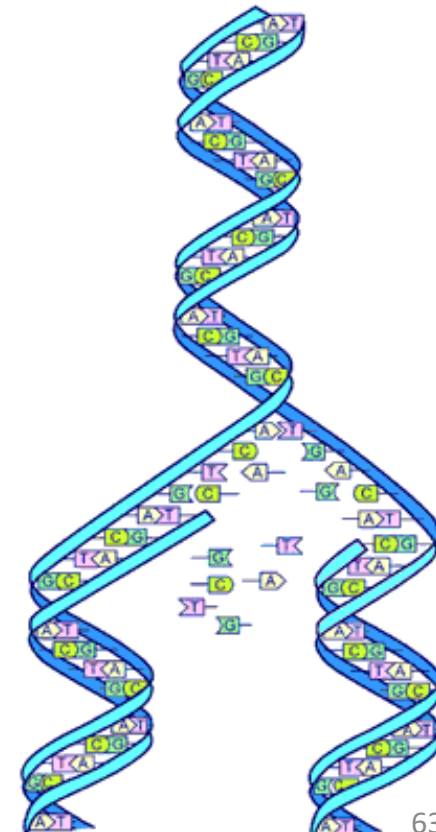
C++ inside Human Genome Sequencing Project



CELERA
an Applera Corporation Business



Human Genome Sequencing Project



C++ inside James Webb Space Telescope

C++ inside James Webb Space Telescope



- The Jet Propulsion Laboratory (JPL)
- The JPL is owned by NASA and managed by the nearby California Institute of Technology (Caltech) for NASA.

- <https://www.jpl.nasa.gov/>

C++ inside James Webb Space Telescope



- The Jet Propulsion Laboratory (JPL)
- The JPL is owned by NASA and managed by the nearby California Institute of Technology (Caltech) for NASA.

- <https://www.jpl.nasa.gov/>

- James Webb is the successor to the Hubble Space Telescope.

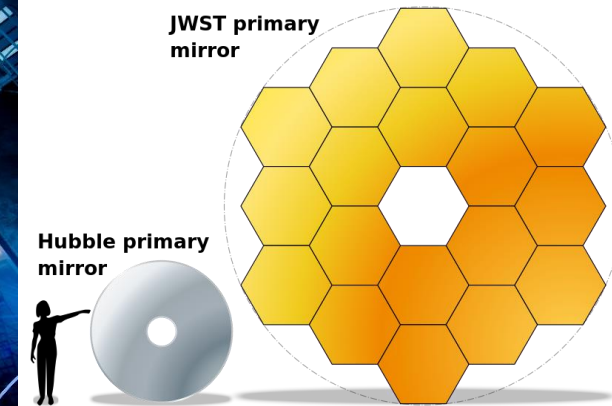
C++ inside James Webb Space T



- The Jet Propulsion Laboratory (JPL)
- The JPL is owned by NASA and managed by the nearby California Institute of Technology (Caltech) for NASA.

<https://www.jpl.nasa.gov/>

- James Webb is the successor to the Hubble Space Telescope.



What is a programming language?

- A notation for algorithms
 - A means for expressing high-level design
 - A way of expressing relationships among concepts
 - A tool for experimentation
 - A means for controlling computer controlled gadgets
 - A mean for controlling computerized devices
 - A tool for instructing machines
 - A means for communications among programmers
 - All of the above!
 - and more
-
- Is study of programming language important?
 - Yes! You can't learn to program without a programming language.

The prehistory of C++

The prehistory of C++

- Bjarne Stroustrup



The prehistory of C++

- Bjarne Stroustrup
- Computing Laboratory at Cambridge university, 1977
- Ph.D. Thesis: Organization of system software for distributed systems.
Simulating software running on a distributed system.



The prehistory of C++

- Bjarne Stroustrup
- Computing Laboratory at Cambridge university, 1977
- Ph.D. Thesis: Organization of system software for distributed systems.
Simulating software running on a distributed system.
- A simulator for simulating software running on a distributed system.
- The simulator was written using *Simula* programming language.



The **S**imula programming language

The **S**imula programming language

- It is based on Algol-60.
- Simula: discrete event simulation language.

The **S**imula programming language

- It is based on Algol-60.
- Simula: discrete event simulation language.
- Class concept: mapping application concepts into language constructs.
Example: class computer

The Simula programming language

- It is based on Algol-60.
- Simula: discrete event simulation language.
- Class concept: mapping application concepts into language constructs.
Example: class computer
- Class hierarchies were used to express variants of application-level concepts.
Example: different types of computers derived from class Computer.

The Simula programming language

- It is based on Algol-60.
- Simula: discrete event simulation language.
- Class concept: mapping application concepts into language constructs.
Example: class computer
- Class hierarchies were used to express variants of application-level concepts.
Example: different types of computers derived from class Computer.
- Concurrency mechanism: Pseudo-parallel Co-routine.

The Simula programming language

- It is based on Algol-60.
- Simula: discrete event simulation language.
- Class concept: mapping application concepts into language constructs.
Example: class computer
- Class hierarchies were used to express variants of application-level concepts.
Example: different types of computers derived from class Computer.
- Concurrency mechanism: Pseudo-parallel Co-routine.
- Type system: compiler's ability to catch type errors.
- Simula *flexible* type system vs. Pascal *rigid* and *primitive* strong type system.

The Simula programming language

- It is based on Algol-60.
- Simula: discrete event simulation language.
- Class concept: mapping application concepts into language constructs.
Example: class computer
- Class hierarchies were used to express variants of application-level concepts.
Example: different types of computers derived from class Computer.
- Concurrency mechanism: Pseudo-parallel Co-routine.
- Type system: compiler's ability to catch type errors.
- Simula *flexible* type system vs. Pascal *rigid* and *primitive* strong type system.
- Simula is the first *Object-oriented* programming language.

Simula language vs. Simula implementation

- Scalability problem
- The Simula implementation was geared to small programs and was inherently unsuitable for larger programs.
- Simula implementation:
 - run-time type checking
 - Guaranteed initialization
 - *Concurrency support*
 - *Garbage collection.*

Simula language vs. Simula implementation

- Scalability problem
- The Simula implementation was geared to small programs and was inherently unsuitable for larger programs.
- Simula implementation:
 - run-time type checking
 - Guaranteed initialization
 - *Concurrency support*
 - *Garbage collection.*
- Simula implementation drawbacks:
 - long link-time
 - Poor runtime performance
 - Garbage collector: %80 of runtime was spent in garbage collector.

Simula language vs. Simula implementation

- Scalability problem
- The Simula implementation was geared to small programs and was inherently unsuitable for larger programs.
- Simula implementation:
 - run-time type checking
 - Guaranteed initialization
 - *Concurrency support*
 - *Garbage collection.*
- Simula implementation drawbacks:
 - long link-time
 - Poor runtime performance
 - Garbage collector: %80 of runtime was spent in garbage collector.
- Writing simulator with BCPL programming language.
- BCPL makes C look like a very high-level language and provides absolutely no type checking or run-time support.

Bell Laboratories



Bell Laboratories

Brian Kernighan
Bjarne Stroustrup
Douglas McIlroy
Dennis Ritchie
Ken Thompson



- B, C, BCPL, C++, UNIX, Plan 9

Unix kernel distribution

Unix kernel distribution

- UNIX Kernel Distribution over a network of computers connected by a LAN:
 - How to analyze the network traffic
 - How to modularize the kernel

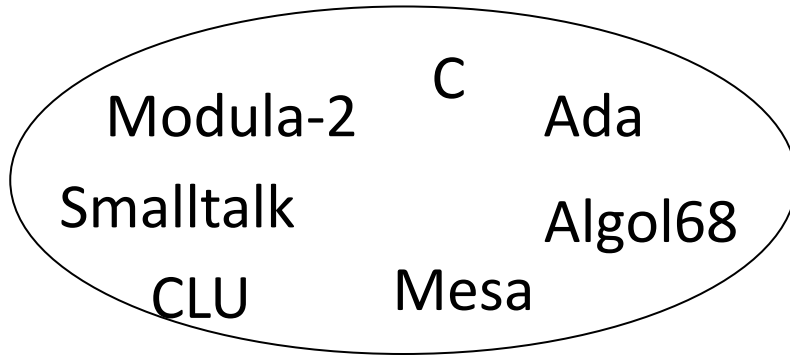
Unix kernel distribution

- UNIX Kernel Distribution over a network of computers connected by a LAN:
 - How to analyze the network traffic
 - How to modularize the kernel
- This is exactly the kind of problem that I had become determined never again to attack without proper tools.

Why C?

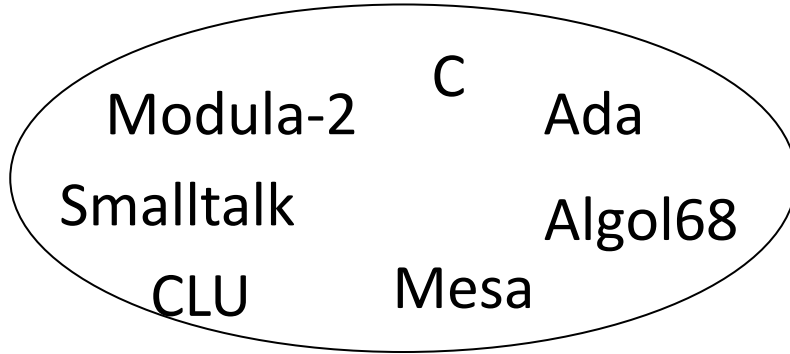
Why C?

Choosing Base programming language:



Why C?

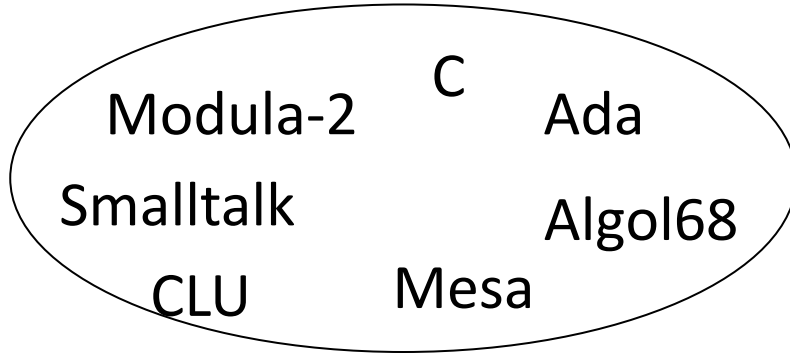
Choosing Base programming language:



- C is clearly not the cleanest language ever designed nor the easiest to use so why do so many people use it?

Why C?

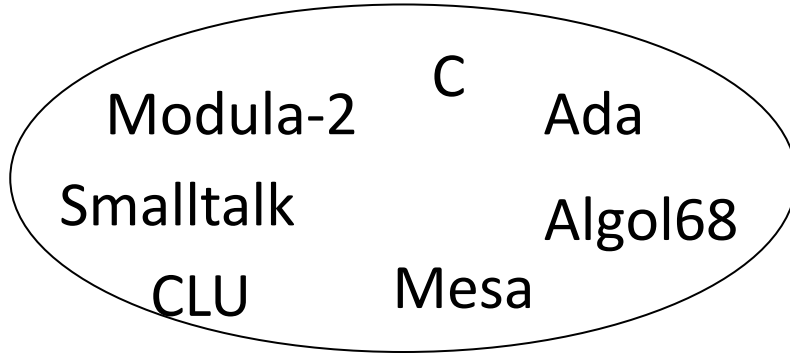
Choosing Base programming language:



- C is clearly not the cleanest language ever designed nor the easiest to use so why do so many people use it?
 - C is *flexible*

Why C?

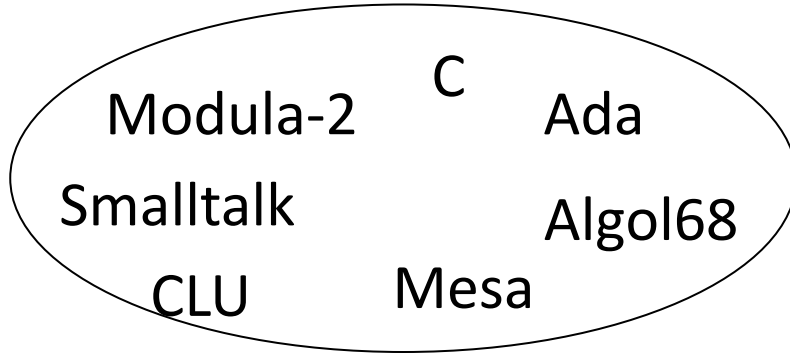
Choosing Base programming language:



- C is clearly not the cleanest language ever designed nor the easiest to use so why do so many people use it?
 - C is *flexible*
 - C is *efficient*

Why C?

Choosing Base programming language:

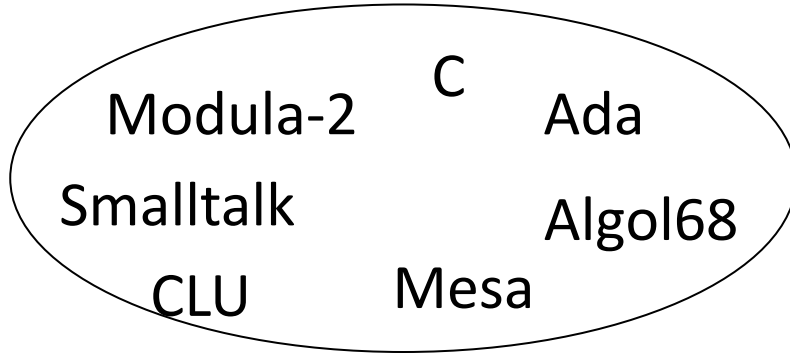


- C is clearly not the cleanest language ever designed nor the easiest to use so why do so many people use it?

- C is *flexible*
- C is *efficient*
- C is *available*

Why C?

Choosing Base programming language:



- C is clearly not the cleanest language ever designed nor the easiest to use so why do so many people use it?

- C is *flexible*
- C is *efficient*
- C is *available*
- C is *portable*

The C programming language?

The C programming language?

C is a small language.

- Brian W. Kernighan & Dennis M. Ritchie: The C Programming Language



The C programming language?

C is a small language.

- Brian W. Kernighan & Dennis M. Ritchie: The C Programming Language



- C : System programming - Procedural programming

The C programming language?

C is a small language.

- Brian W. Kernighan & Dennis M. Ritchie: The C Programming Language



- C : System programming - Procedural programming
- C: machine-architecture-independent notions that directly map to the key hardware notions

The C programming language?

C is a small language.

- Brian W. Kernighan & Dennis M. Ritchie: The C Programming Language



- C : System programming - Procedural programming
- C: machine-architecture-independent notions that directly map to the key hardware notions

Every Valid C program should be a valid C++ program.

The C programming language?

C is a small language.

- Brian W. Kernighan & Dennis M. Ritchie: The C Programming Language



- C : System programming - Procedural programming
- C: machine-architecture-independent notions that directly map to the key hardware notions

Every Valid C program should be a valid C++ program.

- As close to C as Possible, but no closer!

C with classes

C with classes

- October 1979:
 - A preprocessor: Cpre
 - The language accepted by the preprocessor was called “C with Classes”.

C with classes

- October 1979:
 - A preprocessor: Cpre
 - The language accepted by the preprocessor was called “C with Classes”.
- October 1980:
 - Language rather than a tool

C with classes

- October 1979:
 - A preprocessor: Cpre
 - The language accepted by the preprocessor was called “C with Classes”.
- October 1980:
 - Language rather than a tool
 - A General Purpose Language not a Special Purpose one.

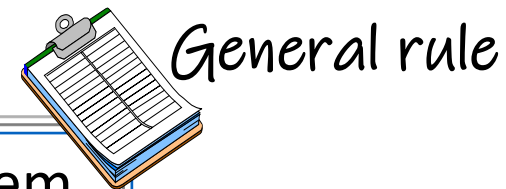
Concurrency, Specific application areas

C with classes

- October 1979:
 - A preprocessor: Cpre
 - The language accepted by the preprocessor was called “C with Classes”.
- October 1980:
 - Language rather than a tool
 - A General Purpose Language not a Special Purpose one.
Concurrency, Specific application areas
 - No compromise between Efficiency and Elegance.

C with classes

- October 1979:
 - A preprocessor: Cpre
 - The language accepted by the preprocessor was called “C with Classes”.
- October 1980:
 - Language rather than a tool
 - A General Purpose Language not a Special Purpose one.
Concurrency, Specific application areas
 - No compromise between Efficiency and Elegance.



General rule

C++ is a language not a complete system.

Original C++ Semantics

Original C++ Semantics

- C++ was designed to combine the strengths of C as a systems programming language with Simula's facilities for organizing programs.



Dennis Ritchie
1941-2011



Kristen Nygaard
1926-2002

Original C++ Semantics

- C++ was designed to combine the strengths of C as a systems programming language with Simula's facilities for organizing programs.



Dennis Ritchie
1941-2011



Kristen Nygaard
1926-2002

- C Semantic: Close to the machine, Machine level facilities, *efficient* code

Original C++ Semantics

- C++ was designed to combine the strengths of C as a systems programming language with Simula's facilities for organizing programs.



Dennis Ritchie
1941-2011



Kristen Nygaard
1926-2002

- C Semantic: Close to the machine, Machine level facilities, *efficient* code
- Simula is the first object-oriented programming language.
- Simula Semantic: close to the problem to be solved, abstraction, *elegant* code

Original C++ Semantics

- C++ was designed to combine the strengths of C as a systems programming language with Simula's facilities for organizing programs.



Dennis Ritchie
1941-2011



Kristen Nygaard
1926-2002

- C Semantic: Close to the machine, Machine level facilities, *efficient* code
- Simula is the first object-oriented programming language.
- Simula Semantic: close to the problem to be solved, abstraction, *elegant* code
- No compromise between efficiency & elegance.
- C++ vs. C: As close to C as possible but no closer!

C onventional definition

Conventional definition

- Many C++ design decisions have their roots in my dislike for forcing people to do things in some particular way.

- Bjarne Stroustrup



Conventional definition

- Many C++ design decisions have their roots in my dislike for forcing people to do things in some particular way.

- Bjarne Stroustrup



1997-2010

C++ is a *general-purpose programming language* with a bias towards *systems programming* that

- is a better C
- supports data abstraction
- supports object-oriented programming
- supports generic programming

Conventional definition

- Many C++ design decisions have their roots in my dislike for forcing people to do things in some particular way.

- Bjarne Stroustrup



1997-2010



C++ is a *general-purpose programming language* with a bias towards *systems programming* that



- is a better C
- supports data abstraction
- supports object-oriented programming
- supports generic programming

Conventional definition

- Many C++ design decisions have their roots in my dislike for forcing people to do things in some particular way.

- Bjarne Stroustrup



1997-2010



C++ is a *general-purpose programming language* with a bias towards *systems programming* that

- is a better C —————> Macros, structures & functions
- supports data abstraction —————> Classes
- supports object-oriented programming —————> Inheritance & Polymorphism
- supports generic programming —————> Templates



Conventional definition

- Many C++ design decisions have their roots in my dislike for forcing people to do things in some particular way.

- Bjarne Stroustrup



1997-2010



C++ is a *general-purpose programming language* with a bias towards *systems programming* that

- is a better C → Macros, structures & functions
- supports data abstraction → Classes
- supports object-oriented programming → Inheritance & Polymorphism
- supports generic programming → Templates



C++ general rule:



General rule

C++ is a language not a complete system.

Conventional definition



- Many C++ design decisions have their roots in my dislike for forcing people to do things in some particular way.

- Bjarne Stroustrup

1997-2010



C++ is a *general-purpose programming language* with a bias towards *systems programming* that

- is a better C —————> Macros, structures & functions
- supports data abstraction —————> Classes
- supports object-oriented programming —————> Inheritance & Polymorphism
- supports generic programming —————> Templates



C++ general rule:



General rule

C++ is a language not a complete system.



- C++ is a multi-paradigm/multi-style programming language.
- It's old, but still very useful definition.

R

References and further readings

- [KR78] Brian W. Kernighan & Dennis M. Ritchie. The C Programming Language, Prentice-Hall, Englewood Cliffs, NJ, 1978.
- [Str94] Bjarne Stroustrup. The Design and Evolution of C++. Addison-Wesley, 1994.
- [Str00] Bjarne Stroustrup. The C++ Programming Language (Special Edition). Addison Wesley. Reading Mass. USA. February 2000.

Thanks for your patience ...

A man who asks a question is a fool for minute,
The man who does not ask, is a fool for a life.
- Confucius

Learning to ask the right (often hard) questions is an essential part of learning to think as a programmer.

- Bjarne Stroustrup *programming Principles and Practice Using C++, page 4.*

There is no stupid question, but there is stupid answer.
- Howard Hinnant

